



**ECOLE MAROCAINE DES
SCIENCES DE L'INGENIEUR**

Membre de **HONORIS UNITED UNIVERSITIES**

Rapport — Projet de Fin d'Année (PFA)

Mini-SOC Web

Surveillance et Détection des Attaques SQLi, XSS et Brute Force

EL KARKOURI Anas

BABBA Omar

AJELYAN Ilyas

EL-ALLAGUI Fadoua

GAGA Omar

4^{ème} ANNÉE INGÉNIERIE INFORMATIQUE ET RÉSEAU (4IIR)

Filière : Cybersécurité

Encadrant : Pr. Ait Lasri Ismail

Année Universitaire : 2025 – 2026

Remerciements

*Nous tenons à exprimer notre profonde gratitude à notre encadrant, **Pr. Ait Lasri Ismail**, pour son accompagnement, ses conseils avisés et sa disponibilité tout au long de ce projet. Ses remarques constructives et son expertise en cybersécurité ont permis d'orienter nos travaux et d'atteindre des résultats conformes aux exigences académiques et professionnelles de la filière.*

*Nous remercions vivement l'**EMSI (École Marocaine des Sciences de l'Ingénieur)** pour la formation de qualité dispensée, les ressources pédagogiques mises à notre disposition et l'environnement propice au développement de nos compétences en ingénierie informatique et en cybersécurité.*

*Nous adressons nos sincères remerciements aux membres de notre équipe : **Anas El Karkouri, Omar Babba, Fadoua El-Allagui, Ilyas Ajelyan** et **Omar Gaga**, pour leur collaboration, leur esprit d'équipe et leur engagement tout au long de ce projet. La réussite de ce travail est le fruit d'un effort collectif et d'une synergie constante entre chacun d'entre nous.*

Enfin, nous souhaitons exprimer notre reconnaissance envers nos familles pour leur soutien indéfectible, leur patience et leurs encouragements qui nous ont été précieux durant toute cette année universitaire.

Résumé

Le présent projet de fin d'année, intitulé **Mini-SOC Web**, s'inscrit dans le cadre de la formation en cybersécurité de l'École Marocaine des Sciences de l'Ingénieur (EMSI). Son objectif principal est de concevoir et de développer une solution pédagogique complète et accessible qui simule un Centre d'Opérations de Sécurité (SOC) miniature, capable de détecter et d'enrichir automatiquement les trois attaques web les plus critiques : l'injection SQL (SQLi), le cross-site scripting (XSS) et les attaques par force brute. La méthodologie adoptée repose sur une architecture modulaire composée de cinq éléments : une application web vulnérable (Atlas MiniShop) servant de terrain d'expérimentation, un collecteur de logs HTTP pour l'acquisition des événements, un moteur de détection Mini-SIEM basé sur des règles regex, un dashboard SOC opérationnel pour la visualisation et la gestion des alertes, et un module de mapping MITRE ATT&CK; pour l'enrichissement académique et professionnel des alertes. Les résultats obtenus démontrent une détection automatisée efficace des trois types d'attaques, la génération d'alertes enrichies avec les identifiants MITRE, les tactiques et les réponses recommandées, ainsi que la production de rapports PDF de synthèse directement depuis le tableau de bord. Le projet a été développé en équipe de cinq membres sur une période de huit semaines, avec une répartition claire des tâches et une démarche de validation rigoureuse. En conclusion, le Mini-SOC Web constitue un outil pédagogique complet, répliquable et extensible, offrant aux étudiants une vision à 360 degrés du cycle de vie d'un incident de sécurité, de l'attaque initiale jusqu'à la remédiation documentée.

Table des matières

Remerciements

Résumé

Introduction générale

Chapitre 1 : Problématique

- 1.1 Constat de départ
 - 1.1.1 L'absence de surveillance dans les petites structures
 - 1.1.2 Des vecteurs d'attaque à faible barrière d'entrée
 - 1.1.3 Le manque d'outils pédagogiques adaptés
- 1.2 La question centrale du projet
- 1.3 Sous-questions opérationnelles

Chapitre 2 : Plan de Travail et Répartition des Tâches

- 2.1 Objectifs du projet
- 2.2 Planification du projet
- 2.3 Répartition des tâches
- 2.4 Dépendances et flux de collaboration

Chapitre 3 : État de l'Art et Contexte

- 3.1 Les Centres d'Opérations de Sécurité (SOC)
- 3.2 Solutions SOC commerciales
- 3.3 Solutions SOC open-source
- 3.4 Positionnement du Mini-SOC Web

Chapitre 4 : Présentation du Projet et Architecture

- 4.1 Contexte et motivation
- 4.2 Les modules du projet
- 4.3 Flux de données complet
- 4.4 Stack technologique

3.5 Diagramme de cas d'utilisation (Use Case)

4.6 Diagramme de séquence

4.7 Cadre éthique et légal

4.7.1 Considérations éthiques

4.7.2 Cadre légal

4.7.3 Bonnes pratiques de sécurité

Chapitre 5 : Les Attaques Web

5.1 Injection SQL (SQLi)

5.1.1 Mécanismes et principes

5.1.2 Variantes et techniques d'injection

5.1.3 Impacts et risques

5.1.4 Table de détection SQLi

5.1.5 Contre-mesures

5.2 Cross-Site Scripting (XSS)

5.2.1 Mécanismes et principes

5.2.2 Variantes d'XSS

5.2.3 Impacts et risques

5.2.4 Table de détection XSS

5.2.5 Contre-mesures

4.3 Attaque par force brute (Brute Force)

5.3.1 Mécanismes et principes

4.3.2 Impacts et risques

5.3.3 Table de détection Brute Force

5.3.4 Contre-mesures

5.4 Référencement MITRE ATT&CK;

Chapitre 6 : Conception et Développement

6.1 Module M1 — Application Atlas MiniShop

6.1.1 Architecture de l'application

6.1.2 Vulnérabilités intentionnelles

6.1.3 Fonctionnalités e-commerce

6.2 Module M2 — Collecteur de logs HTTP

6.2.1 Principe de fonctionnement

6.2.2 Structure de la table http_logs

6.2.3 API REST du dashboard

6.3 Module M3 — Mini-SIEM Python

6.3.1 Architecture du moteur de détection

6.3.2 Règles de détection

6.3.3 Système de scoring des menaces

6.3.4 Enrichissement MITRE

6.4 Module M4 — Dashboard SOC

6.4.1 Architecture du dashboard

6.4.2 Vue d'ensemble (index.php)

6.4.3 Historique et filtres (alerts.php)

6.4.4 Génération de rapports PDF (report.php)

6.4.5 Guide d'utilisation (guide.php)

5.5 Module M5 — Mapping MITRE ATT&CK;

5.5.1 Structure du fichier mitre_mapping.json

5.5.2 Exemple de mapping pour l'injection SQL

5.6 Base de données

5.6.1 Schéma relationnel

5.6.2 Table alerts

Chapitre 7 : Tests et Validation

6.1 Protocole de test

6.2 Tests d'injection SQL

6.2.1 Test sur la barre de recherche

6.2.2 Test sur le formulaire de login

6.2.3 Test de fuite d'informations par erreur SQL

6.2.4 Validation de la détection

6.3 Tests de cross-site scripting

6.3.1 Test XSS réfléchi sur la recherche

6.3.2 Test XSS stocké sur les avis

6.3.3 Validation de la détection XSS

6.4 Tests de force brute

6.4.1 Test de tentatives répétées

7.4.2 Validation de la détection Brute Force

7.5 Validation du cycle complet

Chapitre 8 : Dashboard SOC et Visualisation

8.1 Vue d'ensemble du dashboard

8.2 Statistiques et graphiques

8.2.1 Cartes de statistiques

8.2.2 Graphique de tendance temporelle

8.2.3 Diagramme de répartition par gravité

8.3 Fiches de détail d'alerte

8.3.1 Structure du panneau de détail

8.3.2 Informations MITRE intégrées

8.4 Rapports PDF et documentation

8.4.1 Génération de rapports natifs

8.4.2 Guide d'utilisation intégré

8.5 Configuration et personnalisation

8.5.1 Paramètres utilisateur

8.5.2 Mode confidentialité

Conclusion générale

Bibliographie

Table des figures

Figure 4.1 : Diagramme de cas d'utilisation — Acteurs et interactions avec le Mini-SOC Web

Figure 4.2 : Diagramme de séquence — Scénario d'injection SQL du payload à l'alerte MITRE

Figure 8.1 : Page d'accueil d'Atlas MiniShop — Catalogue des produits en vedette

Figure 8.2 : Page Catégories d'Atlas MiniShop — Navigation par catégories de produits

Figure 8.3 : Page Catalogue produits — Liste des produits avec filtres

Figure 8.4 : Page de connexion — Formulaire d'authentification vulnérable

Figure 8.5 : Page Connexion client — Espace client avec profil et historique

Figure 8.6 : Page produit avec formulaire d'avis — Surface d'attaque XSS stocké

Figure 8.7 : Page produit avec avis — Détail d'un produit et avis clients

Figure 8.8 : Back-office administrateur — Gestion des commandes et avis

Figure 7.9 : Espace client — Profil et historique des commandes

Figure 8.10 : Page Suivi de commande — Formulaire de suivi par code de commande

Figure 8.11 : Résultat suivi commande — Détail de la commande et statut de livraison

Figure 8.12 : Page Panier — Récapitulatif des articles avant commande

Figure 8.13 : Structure de la base de données minisoc_shop sous phpMyAdmin

Figure 8.14 : Table http_logs — Logs des requêtes HTTP collectées

Figure 8.15 : Table alerts — Alertes enrichies avec informations MITRE

Figure 8.16 : Table alerts détaillée phpMyAdmin — Vue complète des alertes avec MITRE

Figure 8.17 : Page de connexion du dashboard SOC — Authentification sécurisée

Figure 8.18 : Vue globale du dashboard SOC — Statistiques et alertes en temps réel

Figure 8.1 : Injection SQL dans la barre de recherche — payload ' OR '1'='1' retournant tout le catalogue

Figure 8.2 : Payload SQLi sur le login — admin' OR 1=1#

Figure 8.3 : Connexion réussie en tant qu'administrateur après injection SQL

Figure 8.4 : Fuite d'information par erreur SQL — requête complète affichée en clair

Figure 8.5 : XSS réfléchi dans la barre de recherche — payload alert('XSS')

Figure 8.6 : Résultat XSS réfléchi dans le navigateur — Exécution du payload JavaScript

Figure 8.7 : XSS stocké — formulaire d'avis avec payload alert('XSS')

Figure 8.8 : Table reviews sous phpMyAdmin — scripts XSS visibles en base de données

Figure 7.9 : Tentatives de force brute — outils de développement montrant les réponses 401 répétées

Figure 8.1 : Vue globale du dashboard SOC avec statistiques, graphiques et alertes récentes

Figure 8.2 : Panneau de détail d'alerte avec informations MITRE complètes

Figure 8.3 : Vue détaillée dashboard avec filtres et panneau de détail d'alerte

Figure 8.4 : Timeline des attaques — Graphique empilé montrant la répartition temporelle

Figure 8.5 : Page MITRE ATT&CK; — Mapping complet des trois types d'attaque

Figure 8.6 : Interface de génération de rapport PDF — Paramètres et prévisualisation

Figure 8.7 : Guide d'utilisation — Processus de gestion des incidents en 4 étapes

Figure 8.8 : Réponses recommandées MITRE — Actions correctives pour chaque attaque

Introduction générale

Dans un monde de plus en plus connecté et dépendant des technologies numériques, la sécurité des systèmes d'information est devenue un enjeu stratégique majeur pour les entreprises, les administrations et les particuliers. Les cyberattaques se multiplient en nombre et en sophistication, exploitant les vulnérabilités des applications web, qui représentent aujourd'hui la surface d'attaque la plus exposée sur Internet. Selon le rapport de l'OWASP Top 10, les injections SQL, les failles XSS et les attaques par force brute demeurent parmi les menaces les plus récurrentes et les plus critiques pour la sécurité des applications web.

Face à cette menace croissante, les organisations déploient des Centres d'Opérations de Sécurité (SOC) chargés de surveiller en permanence leur infrastructure, de détecter les incidents de sécurité et d'y répondre de manière efficace. Cependant, la mise en place d'un SOC professionnel complet nécessite des investissements considérables en termes d'infrastructure, de logiciels et de compétences humaines, ce qui le rend inaccessible dans un contexte académique ou pour de petites structures.

C'est dans ce contexte que s'inscrit notre projet de fin d'année intitulé **Mini-SOC Web**. Ce projet a pour ambition de concevoir et de développer une solution pédagogique complète qui simule, à échelle réduite, l'ensemble de la chaîne de détection et de réponse aux incidents de sécurité. Le projet se décompose en quatre modules complémentaires : une application web vulnérable servant de terrain d'expérimentation, un collecteur de logs responsable de l'acquisition des événements, un moteur de détection basé sur des règles d'analyse, et un tableau de bord de visualisation des alertes.

L'objectif pédagogique principal est de permettre une compréhension approfondie du cycle de vie d'une opération de sécurité, depuis l'attaque initiale jusqu'à la remédiation, en passant par la détection, l'analyse et la documentation. Les trois vecteurs d'attaque retenus — l'injection SQL, le cross-site scripting et la force brute — ont été choisis pour leur représentativité et leur fréquence dans les incidents réels. Chaque attaque est enrichie par un mapping avec le framework MITRE ATT&CK, assurant ainsi une dimension académique et professionnelle au projet.

Le présent rapport présente l'intégralité du travail réalisé au cours de ce projet. Il débute par la présentation du plan de travail et de la répartition des tâches au sein de l'équipe, puis aborde l'état de l'art des solutions SOC existantes. Les chapitres suivants détaillent l'architecture du projet, la théorie des attaques web étudiées, la conception et le développement de chaque module, les phases de test et de validation, ainsi que la présentation du tableau de bord SOC. Enfin, une conclusion générale dresse le bilan des réalisations et envisage les perspectives d'amélioration.

Nous tenons à exprimer notre profonde gratitude à notre encadrant, **Pr. Ait Lasri Ismail**, pour son accompagnement, ses conseils avisés et sa disponibilité tout au long de ce projet. Ses remarques constructives ont permis d'orienter nos travaux et d'atteindre des résultats conformes aux exigences académiques et professionnelles de la filière cybersécurité de l'École Marocaine des Sciences de l'Ingénieur (EMSI).

Chapitre 1 : Problématique

1.1 Constat de départ

1.1.1 L'absence de surveillance dans les petites structures

La grande majorité des cyberattaques qui réussissent ne sont pas détectées immédiatement. Selon le rapport IBM Cost of a Data Breach 2023, le délai moyen entre une intrusion et sa détection est de 204 jours. Ce chiffre alarmant révèle un problème fondamental : les organisations, et en particulier les petites structures, ne disposent pas de mécanismes de surveillance adaptés.

Dans le contexte des PME et startups marocaines, cette réalité est encore plus marquée :

- Absence quasi-totale de SOC (Centre Opérationnel de Sécurité)
- Pas de personnel dédié à la surveillance de la sécurité
- Applications web déployées sans audit de sécurité préalable
- Logs d'accès non surveillés (ou même non configurés)
- Budget cybersécurité quasi-inexistant dans les petites structures

1.1.2 Des vecteurs d'attaque à faible barrière d'entrée

Les trois attaques ciblées par notre projet (SQLi, XSS, Brute Force) partagent une caractéristique commune : elles sont accessibles à des attaquants peu expérimentés.

De nombreux outils automatisés (SQLMap pour SQLi, Hydra pour le Brute Force) permettent à un attaquant novice de lancer des attaques sophistiquées sans compétences avancées.

Combinée à l'absence de surveillance, cette réalité crée une situation dangereuse : des applications web vulnérables, exposées sur Internet, attaquées par des outils automatisés, sans aucune alerte générée.

1.1.3 Le manque d'outils pédagogiques adaptés

Du côté académique, les étudiants en cybersécurité apprennent souvent les attaques web de manière théorique ou via des plateformes externes (TryHackMe, HackTheBox). Cependant, très peu de projets permettent de vivre le cycle complet : attaque -> log -> détection -> alerte -> réponse.

Les solutions existantes (comme ELK Stack ou Wazuh) sont trop complexes pour être déployées dans un cadre académique débutant. Il manque un outil simple, complet et explicatif qui permette aux étudiants de comprendre concrètement comment fonctionne la détection des attaques web.

1.2 La question centrale du projet

Question Centrale du Projet :

Comment concevoir et implémenter un système de surveillance de sécurité web léger et pédagogique, capable de détecter en temps réel les attaques SQLi, XSS et Brute Force, d'alerter l'administrateur de manière exploitable, et de servir de support d'apprentissage concret ?

1.3 Sous-questions opérationnelles

Pour répondre à cette question centrale, nous avons identifié 5 sous-questions opérationnelles qui guident l'ensemble de notre démarche de conception.

Q1 — Collecte des événements : Comment collecter de manière exhaustive et structurée les événements HTTP générés par l'application web, en préservant toutes les informations nécessaires à l'analyse (IP source, URL, méthode, paramètres, timestamp) ?

Q2 — Précision des règles de détection : Comment définir des règles de détection (patterns regex) suffisamment précises pour identifier les attaques SQLi, XSS et Brute Force, tout en minimisant les faux positifs et les faux négatifs ?

Q3 — Exploitabilité des alertes : Comment présenter les alertes générées dans un tableau de bord suffisamment clair et informatif pour qu'un administrateur, même peu expérimenté, puisse comprendre la nature de l'incident et savoir comment réagir ?

Q4 — Intégration MITRE ATT&CK : Comment intégrer le référentiel MITRE ATT&CK; dans notre système de manière à enrichir les alertes avec des informations contextuelles (technique utilisée, tactique associée) utiles pour la compréhension et la formation ?

Q5 — Validation et mesure de l'efficacité : Comment valider que le système fonctionne correctement et mesurer son efficacité en termes de taux de détection, de temps de réponse et de pertinence des alertes, à travers des scénarios de test reproductibles ?

Chapitre 2 : Plan de Travail et Répartition des Tâches

2.1 Objectifs du projet

Le projet Mini-SOC Web s'articule autour de plusieurs objectifs stratégiques et pédagogiques. Le premier objectif est de concevoir une application web fonctionnelle, représentative d'un site de commerce électronique, mais volontairement vulnérable aux trois attaques principales étudiées : l'injection SQL, le cross-site scripting et la force brute. Cette application, nommée Atlas MiniShop, sert de terrain d'expérimentation contrôlé et réaliste.

Le deuxième objectif consiste à mettre en place un mécanisme de collecte et de stockage des événements HTTP sous forme de logs structurés. Chaque requête adressée à l'application vulnérable est interceptée, analysée et enregistrée dans une base de données relationnelle, fournissant ainsi une source de données brute pour l'analyse ultérieure.

Le troisième objectif vise à développer un moteur de détection automatisé, inspiré des fonctionnalités d'un SIEM (Security Information and Event Management). Ce moteur, implémenté en Python, analyse les logs collectés à l'aide de règles basées sur des expressions régulières et des seuils comportementaux, identifie les tentatives d'attaque et génère des alertes enrichies par le référentiel MITRE ATT&CK;

Le quatrième objectif est la création d'un tableau de bord d'administration sécurisé, permettant à un analyste SOC de visualiser les alertes en temps réel, d'accéder aux détails techniques de chaque incident, de consulter les recommandations de remédiation et de générer des rapports de synthèse. Ce dashboard constitue l'interface opérationnelle du Mini-SOC.

Enfin, le dernier objectif est de garantir la cohérence et l'intégration de l'ensemble des modules, de manière à ce que le cycle complet — de l'attaque à la remédiation — soit fonctionnel, reproductible et démontrable dans un contexte académique.

2.2 Planification du projet

La planification du projet a été organisée sur une durée totale de huit semaines, réparties en sept phases distinctes mais interconnectées. La méthodologie adoptée privilégie une approche séquentielle avec des chevauchements progressifs, permettant à chaque membre de l'équipe de débiter son module dès que les livrables du module précédent sont disponibles. Le diagramme de Gantt ci-dessous présente cette planification de manière synthétique.

Tableau : Diagramme de Gantt simplifié

Phase	S1	S2	S3	S4	S5	S6	S7	S8
P1 — Cadrage	■							
P2 — État de l'Art		■						
P3 — App Vulnérable			■					
P4 — Collecteur Logs			■	■				
P5 — Mini-SIEM + MITRE				■	■			
P6 — Dashboard					■	■		
P7 — Tests + Rédaction							■	■

La première semaine (S1) est consacrée au cadrage général du projet, à la définition des objectifs, à la sélection du stack technique et à la conception de l'architecture globale. La semaine 2 (S2) est dédiée à l'état de l'art et à l'analyse des solutions SOC existantes. Les semaines 3 et 4 (S3-S4) voient le développement en parallèle de l'application vulnérable (P3) et du collecteur de logs (P4), qui constituent le socle sur lequel reposent tous les autres modules. Les semaines 4, 5 et 6 (S4-S6) sont consacrées au développement du Mini-SIEM avec mapping MITRE (P5) et du dashboard SOC (P6), en parallèle et avec des chevauchements progressifs. Enfin, les semaines 7 et 8 (S7-S8) sont réservées aux tests end-to-end, à la validation du cycle complet et à la rédaction du rapport final (P7).

2.3 Répartition des tâches

La répartition des tâches au sein de l'équipe a été pensée pour garantir une charge de travail équilibrée tout en capitalisant sur les compétences spécifiques de chaque membre. Cinq rôles principaux ont été définis, correspondant aux cinq modules du projet. Chaque membre dispose d'une fiche de mission détaillée, d'un livrable attendu et de critères de validation explicites. La collaboration entre les membres est formalisée par des dépendances fonctionnelles, assurant une progression fluide et une intégration harmonieuse des différents composants.

Anas El Karkouri — Chef de projet et coordination générale

Anas El Karkouri assume la responsabilité de chef de projet, assurant la coordination globale, le suivi de l'avancement et la planification des activités. Il est chargé de l'élaboration du planning détaillé, de l'organisation des réunions de suivi et de la gestion des livrables. En parallèle, il développe l'application web vulnérable Atlas MiniShop (M1), conçoit la structure de la base de données, implémente les pages PHP volontairement vulnérables et s'assure que les mécanismes de collecte de logs sont correctement intégrés. Il réalise également l'état de l'art du projet, en analysant les solutions SOC existantes et en positionnant le Mini-SOC Web par rapport aux offres du marché. Enfin, il rédige le rapport final, coordonne la démonstration de soutenance et assure la validation finale du cycle complet de détection. Ses compétences en développement web et en gestion de projet ont été déterminantes pour la réussite de l'ensemble des livrables.

Omar Babba — Collecteur de logs HTTP (M2)

Omar Babba est responsable du module de collecte des événements HTTP (M2). Sa mission consiste à concevoir et à développer le mécanisme d'interception et de stockage des requêtes adressées à l'application Atlas MiniShop. Il implémente le script de collecte en PHP, qui capture l'adresse IP source, la méthode HTTP, l'URL cible, les paramètres GET et POST, le User-Agent et le code de réponse HTTP. Les données sont structurées au format JSON et stockées dans la table dédiée `http_logs` de la base MySQL. Omar s'assure que chaque action importante sur le site génère un log complet, lisible et exploitable par le moteur de détection. Il développe également les API REST nécessaires au fonctionnement du dashboard et rédige la documentation technique du module M2. Ses tests de validation démontrent que les requêtes de recherche, de connexion et de soumission d'avis produisent bien des entrées exploitables dans la table des logs.

Fadoua El-Allagui — Moteur de détection Mini-SIEM (M3)

Fadoua El-Allagui est responsable du moteur de détection et d'analyse des logs, cœur du Mini-SOC (M3). Elle développe le script Python `mini_siem.py`, qui se connecte périodiquement à la base de données, lit les logs non traités, et applique un ensemble de règles de détection basées sur des expressions régulières. Ces règles couvrent les trois types d'attaques : l'injection SQL, le cross-site scripting et la force brute. Fadoua définit les niveaux de sévérité (LOW, MEDIUM, HIGH, CRITICAL) et implémente le mécanisme de classification automatique des alertes. Elle conçoit la structure de la table `alerts`, qui stocke les alertes enrichies avec les informations MITRE. Son travail est coordonné avec celui d'Ilyas Ajelyan, qui lui fournit le mapping MITRE ATT&CK. Elle rédige la documentation des règles de détection et effectue des tests unitaires pour valider que chaque payload suspect génère une alerte correctement enrichie.

Ilyas Ajelyan — Mapping MITRE ATT&CK; et enrichissement (M5)

Ilyas Ajelyan est responsable de la dimension académique et professionnelle du projet à travers le mapping MITRE ATT&CK; (M5). Il établit la correspondance entre les trois attaques étudiées et le référentiel MITRE, identifiant les identifiants de technique (T1190 pour l'injection SQL, T1059.007 pour le XSS et T1110 pour la force brute), les tactiques associées (Initial Access, Execution, Credential Access) et les indicateurs de compromission (IoC) pertinents. Il rédige le fichier `mitre_mapping.json`, qui sert de base de connaissances au moteur de détection. Ce fichier contient, pour chaque attaque, la description technique, les patterns de détection, les seuils de déclenchement, les réponses recommandées et les étapes d'investigation. Ilyas travaille en étroite collaboration avec Fadoua pour intégrer ces informations dans les alertes générées par le Mini-SIEM. Il prépare également les scénarios de test de référence et la matrice de sévérité utilisée par le système de scoring.

Omar Gaga — Dashboard SOC (M4)

Omar Gaga est responsable de la conception et du développement du tableau de bord d'administration SOC (M4). Il développe l'interface web complète, indépendante du back-office e-commerce, qui permet à l'administrateur de visualiser les alertes en temps réel, de consulter les statistiques globales et de générer des rapports de synthèse. Il implémente les cartes de statistiques (nombre total d'alertes, répartition par type d'attaque, score global de menace), le tableau des alertes récentes avec filtres dynamiques (par type, par gravité, par statut), la page de détail d'alerte avec panel MITRE, et la fonctionnalité de génération de rapports PDF natifs. Il conçoit également le guide d'utilisation intégré et le système de thème clair/sombre. Omar s'assure que les informations MITRE préparées par Ilyas sont correctement affichées et que le dashboard est simple, lisible et utile pour une démonstration académique de type SOC.

2.4 Dépendances et flux de collaboration

Le bon déroulement du projet repose sur une chaîne de dépendances bien établie. Omar Babba (M2) a besoin que l'application M1 soit stable pour collecter des logs exploitables. Fadoua (M3) dépend des logs de M2 et du mapping MITRE d'Ilyas (M5) pour enrichir ses alertes. Omar Gaga (M4) utilise la structure des alertes de M3 pour afficher les données correctement. Anas (chef de projet) valide l'intégration de bout en bout et s'assure que tous les modules communiquent correctement. Cette organisation en pipeline garantit une progression fluide et minimise les blocages.

Chapitre 3 : État de l'Art et Contexte

3.1 Les Centres d'Opérations de Sécurité (SOC)

Un Centre d'Opérations de Sécurité (Security Operations Center, SOC) est une entité organisationnelle, technologique et humaine chargée de surveiller, détecter, analyser et répondre aux incidents de cybersécurité. Le SOC constitue le cœur nerveux de la défense d'une organisation face aux menaces numériques. Il centralise les flux de logs, les alertes de sécurité et les informations sur les menaces (Threat Intelligence) pour fournir une vision unifiée et temps réel de la posture de sécurité.

Les fonctions principales d'un SOC incluent la surveillance continue du réseau et des systèmes, la détection d'anomalies et d'intrusions, l'analyse et la qualification des alertes, la réponse aux incidents (containment, eradication, recovery), la gestion des vulnérabilités, la production de rapports de conformité et la rétroaction vers les équipes de développement pour corriger les failles. La maturité d'un SOC peut être évaluée selon plusieurs niveaux, allant du SOC de base (principalement réactif) au SOC avancé (proactif, avec intelligence artificielle et orchestration automatique).

L'architecture type d'un SOC repose sur trois piliers : la collecte de données (logs réseau, endpoints, applications, cloud), le traitement et la corrélation (SIEM, EDR, XDR), et la visualisation et la réponse (dashboards, playbooks, SOAR). Les outils modernes intègrent de plus en plus des capacités d'analyse comportementale, de machine learning et de threat hunting pour anticiper les attaques plutôt que de simplement y répondre.

3.2 Solutions SOC commerciales

Le marché des solutions SOC est dominé par plusieurs acteurs majeurs proposant des plateformes intégrées et robustes. **Splunk Enterprise Security** est l'un des leaders incontestés. Cette plateforme offre une capacité d'indexation massive des logs, des tableaux de bord personnalisables, des règles de corrélation avancées et un écosystème d'applications enrichi. Splunk excelle dans la gestion des big data de sécurité mais représente un investissement considérable, tant en termes de licence qu'infrastructure.

IBM QRadar constitue une autre solution entreprise de référence. QRadar intègre nativement un SIEM, une gestion des vulnérabilités et une analyse des flux réseau. Sa force réside dans sa

capacité de corrélation automatique et sa bibliothèque de règles préconfigurées. QRadar est particulièrement apprécié dans les environnements hétérogènes et les grandes organisations.

Microsoft Sentinel est la solution cloud-native de Microsoft, intégrée à l'écosystème Azure. Elle combine les capacités SIEM et SOAR avec des fonctionnalités d'intelligence artificielle et de threat hunting. Sentinel est particulièrement compétitive pour les organisations déjà investies dans Microsoft 365 et Azure, offrant une tarification à l'usage attractive pour les PME.

Ces solutions commerciales, bien que puissantes, présentent des inconvénients majeurs dans un contexte académique : leur coût prohibitif, leur complexité de déploiement, leur courbe d'apprentissage abrupte et leur dépendance à une infrastructure d'entreprise. Elles restent cependant des références incontournables pour comprendre les standards de l'industrie.

3.3 Solutions SOC open-source

Face aux coûts des solutions commerciales, l'écosystème open-source offre des alternatives pertinentes, particulièrement adaptées à la recherche, à l'enseignement et aux petites structures. La stack **ELK (Elasticsearch, Logstash, Kibana)** est la plus répandue pour la centralisation et la visualisation de logs. Elasticsearch assure le stockage et la recherche, Logstash la collecte et le traitement, et Kibana la visualisation. Cette stack est flexible, scalable et bien documentée, mais sa configuration peut être complexe.

Wazuh est une plateforme de sécurité open-source qui combine les fonctionnalités HIDS (Host-based Intrusion Detection System), EDR légère et conformité. Wazuh collecte les logs système, surveille l'intégrité des fichiers, détecte les rootkits et corrèle les événements avec sa propre base de règles. Son intégration avec ELK en fait une solution SOC complète et accessible.

OSSEC est un autre HIDS mature, orienté vers la détection d'intrusions sur les serveurs et les postes de travail. Il est léger, efficace et bien adapté aux environnements Linux. **Graylog** se positionne comme une alternative à la stack ELK, offrant une interface de gestion des logs plus simple et une corrélation intégrée. Enfin, **ModSecurity** est un WAF (Web Application Firewall) open-source qui protège les applications web en inspectant les requêtes HTTP en temps réel.

Les projets pédagogiques comme **DVWA (Damn Vulnerable Web Application)** et **WebGoat** constituent des références pour l'apprentissage des vulnérabilités web. Bien qu'ils ne soient pas des SOC, ils fournissent le terrain d'expérimentation indispensable à la compréhension des mécanismes d'attaque. Notre projet Mini-SOC Web s'inspire de ces initiatives tout en ajoutant la dimension de détection et de réponse.

3.4 Positionnement du Mini-SOC Web

Le Mini-SOC Web se positionne comme une solution pédagogique complète, à l'intersection des solutions open-source et des projets de vulnérabilité. Contrairement aux SIEM commerciaux, il est entièrement gratuit, déployable sur un environnement local (XAMPP/WAMP) et ne nécessite aucune infrastructure cloud. Contrairement à la stack ELK, il privilégie la simplicité : une seule base de données MySQL, un seul script Python, une interface web légère. Contrairement à DVWA, il ne se contente pas de montrer les vulnérabilités : il démontre l'ensemble du cycle de détection, d'enrichissement et de réponse.

Le projet est conçu pour être accessible aux étudiants en cybersécurité, répliquable sans expertise avancée, et extensible pour des travaux futurs. Il couvre trois attaques représentatives, enrichies par le référentiel MITRE ATT&CK, et produit des livrables concrets : logs structurés, alertes classifiées, rapports PDF et guide d'utilisation. Cette approche complète et progressive en fait un outil d'apprentissage idéal pour la filière cybersécurité de l'EMSI.

Chapitre 4 : Présentation du Projet et Architecture

4.1 Contexte et motivation

La filière cybersécurité de l'École Marocaine des Sciences de l'Ingénieur (EMSI) vise à former des ingénieurs capables de concevoir, déployer et administrer des systèmes de sécurité robustes. Dans ce cadre, le projet de fin d'année constitue une opportunité unique pour mettre en pratique les compétences acquises en développement web, en administration de bases de données, en programmation Python et en analyse des menaces. Le choix d'un Mini-SOC Web s'inscrit dans une logique de progression pédagogique : comprendre l'attaque pour mieux la défendre.

La motivation première est de démontrer que la sécurité ne se limite pas à la prévention, mais s'étend à la détection, à l'analyse et à la réponse. Un étudiant en cybersécurité doit non seulement savoir coder une requête préparée pour éviter une injection SQL, mais aussi comprendre comment un analyste SOC détecte cette attaque, la qualifie, la documente et recommande des mesures correctives. Ce projet offre une vision à 360 degrés du cycle de vie d'un incident de sécurité.

Le contexte technique est celui d'un environnement local (localhost) sécurisé, utilisant la stack XAMPP/WAMP. L'application est intentionnellement vulnérable mais isolée : aucun accès depuis l'extérieur n'est autorisé, et des avertissements éthiques sont affichés. Ce cadre garantit que les compétences développées sont utilisées à des fins défensives et pédagogiques uniquement.

4.2 Les modules du projet

Le projet se décompose en quatre modules fonctionnels interconnectés, complétés par un cinquième module de mapping MITRE. Chaque module possède un rôle clairement défini, des livrables précis et des interfaces avec les autres modules.

Module M1 — Application web vulnérable (Atlas MiniShop)

Atlas MiniShop est une application de commerce électronique fictive développée en PHP 8.x avec une base de données MySQL. Elle propose les fonctionnalités classiques d'un site e-commerce : catalogue de produits, recherche, panier, passage de commande, suivi de commande, espace client et back-office administrateur. Cinq pages principales sont volontairement vulnérables : la recherche (SQLi + XSS réfléchi), la page produit avec avis (SQLi + XSS stocké), le login (SQLi + brute force), le suivi de commande (SQLi) et le back-office (SQLi + XSS stocké). Les vulnérabilités sont introduites par l'absence de requêtes préparées et de fonctions d'échappement des sorties HTML. Le

cookie de session est configuré sans attribut `HttpOnly` pour permettre les démonstrations de vol de session via XSS. Cette application sert à la fois de terrain d'expérimentation et de démonstration pédagogique.

Module M2 — Collecteur de logs HTTP

Le collecteur de logs est implémenté sous forme de script PHP inclus dans chaque page de l'application. Il utilise la fonction `register_shutdown_function` pour capturer les requêtes après traitement, assurant ainsi que le code de réponse HTTP final est connu. Pour chaque requête, il extrait : l'adresse IP source (via `X-Forwarded-For` ou `REMOTE_ADDR`), la méthode HTTP, l'URL complète, les paramètres GET et POST sérialisés en JSON, le User-Agent, le code de réponse HTTP et un timestamp précis. Ces données sont insérées dans la table `http_logs` de la base MySQL, avec un champ `processed` initialisé à 0, indiquant que le log n'a pas encore été analysé par le Mini-SIEM. Le collecteur est transparent pour l'utilisateur et n'affecte pas les performances de l'application.

Module M3 — Moteur de détection Mini-SIEM

Le Mini-SIEM est un script Python qui s'exécute en boucle infinie avec un intervalle de dix secondes. Il se connecte à la base MySQL, récupère les logs non traités (`processed = 0`), et applique un ensemble de règles de détection. Chaque règle est définie par une expression régulière (regex) et un niveau de sévérité. Pour l'injection SQL, les règles détectent les apostrophes non échappées, les mots-clés `UNION`, `SELECT`, `DROP`, les constructions `OR 1=1`, les fonctions `SLEEP()` et `BENCHMARK()`, et les appels à `xp_cmdshell`. Pour le XSS, les règles identifient les balises `<script>`, les gestionnaires d'événements `onerror` et `onload`, les protocoles `javascript:`, les appels à `alert()` et `eval()`, et les encodages URL. Pour la force brute, les règles comptent les requêtes POST sur `login.php` depuis une même IP dans une fenêtre temporelle de soixante secondes, détectent les codes 401 consécutifs et identifient les User-Agent suspects (`sqlmap`, `curl`, `Hydra`). Lorsqu'une règle est déclenchée, le Mini-SIEM charge le mapping MITRE d'Ilyas, enrichit l'alerte avec l'ID MITRE, la tactique, la technique et la réponse recommandée, puis insère l'alerte dans la table `alerts`. Le log est marqué comme traité (`processed = 1`) et un score de menace est calculé pour l'IP source.

Module M4 — Dashboard SOC

Le tableau de bord SOC est une application web indépendante, située dans le répertoire `/soc/`, avec son propre système d'authentification (`admin / bcrypt`). Il est développé en PHP avec une interface moderne utilisant CSS3 et JavaScript vanilla. La page principale (`index.php`) affiche une vue d'ensemble avec : le nombre total d'alertes, la répartition par type (SQLi, XSS, Brute Force), le score global de menace, un graphique de tendance temporelle, un diagramme de répartition par gravité, le top 5 des IP attaquantes et un tableau des alertes récentes. L'API interne (`api.php`)

rafraîchit les données toutes les cinq secondes et relance automatiquement le Mini-SIEM s'il est inactif. Les pages annexes incluent : l'historique complet des alertes (alerts.php), le détail d'une alerte avec panel MITRE (detail.php), les paramètres de configuration (settings.php), la génération de rapports PDF (report.php), la documentation MITRE intégrée (mitre.php) et le guide d'utilisation (guide.php). La sécurité du dashboard est renforcée par des tokens CSRF, un fingerprinting de session, une limitation des tentatives de connexion et des en-têtes CSP.

Module M5 — Mapping MITRE ATT&CK;

Le module M5 est constitué par le fichier mitre_mapping.json, qui sert de base de connaissances structurée au projet. Il définit, pour chaque type d'attaque, les attributs MITRE : l'identifiant de technique (T1190, T1059.007, T1110), la tactique associée (Initial Access, Execution, Credential Access), la description de la technique, les indicateurs de compromission (IoC), les regex de détection, les seuils de déclenchement, les réponses immédiates recommandées, les étapes d'investigation, les mesures correctives à long terme et les faux positifs possibles. Ce fichier est chargé par le Mini-SIEM pour enrichir les alertes et par le dashboard pour afficher les informations contextuelles. Il constitue le pont entre la dimension technique du projet et la dimension académique et professionnelle du référentiel MITRE.

4.3 Flux de données complet

Le flux de données du Mini-SOC Web suit une chaîne logique en sept étapes, depuis l'attaque initiale jusqu'à la visualisation par l'analyste. Cette chaîne est illustrée par le schéma conceptuel suivant, décrit en texte structuré pour faciliter la compréhension.

Étape	Composant	Action
1	Attaquant	Envoie une requête malveillante (SQLi, XSS, Brute Force) vers l'application Atlas MiniShop.
2	M1 — App vulnérable	Traite la requête, exécute la requête SQL vulnérable ou affiche le payload, retourne une réponse HTTP.
3	M2 — Collecteur	Intercepte la requête et la réponse, extrait les métadonnées, stocke dans http_logs (processed = 0).
4	M3 — Mini-SIEM	Lit les nouveaux logs, applique les règles regex, détecte l'attaque, génère une alerte enrichie MITRE.
5	Base de données	Stocke l'alerte dans la table alerts avec toutes les informations d'enrichissement.

6	M4 — Dashboard	Récupère les alertes via API REST, affiche les statistiques, les détails et les recommandations.
7	Analyste SOC	Consulte le dashboard, qualifie l'alerte, applique les recommandations et génère un rapport.

4.4 Stack technologique

Couche	Technologie	Justification
Serveur web	Apache 2.4 (XAMPP)	Gratuit, open-source, multiplateforme, intégration native PHP et MySQL.
Langage application	PHP 8.x	Universel, courbe d'apprentissage modérée, intégration native MySQL.
Base de données	MySQL 8.x	Gratuite, relationnelle, excellente intégration PHP et Python.
Moteur d'analyse	Python 3.10+	Polyvalent, bibliothèques re et json natives, connexion MySQL via mysql-connector.
Frontend	HTML5 + CSS3 + Bootstrap 5	Design responsive et professionnel sans expertise CSS avancée.
Visualisation	Chart.js + SVG inline	Légère, bien documentée, graphiques interactifs et tendances.
Sécurité dashboard	CSRF, bcrypt, CSP	Protection contre les attaques classiques sur le panneau d'administration.
Versioning	Git + GitHub	Collaboration, traçabilité, sauvegarde du code source.

3.5 Diagramme de cas d'utilisation (Use Case)

Le diagramme de cas d'utilisation du Mini-SOC Web identifie trois acteurs principaux et sept cas d'utilisation fonctionnels. Cette modélisation permet de comprendre les interactions entre les utilisateurs et le système, ainsi que les frontières fonctionnelles de chaque module.

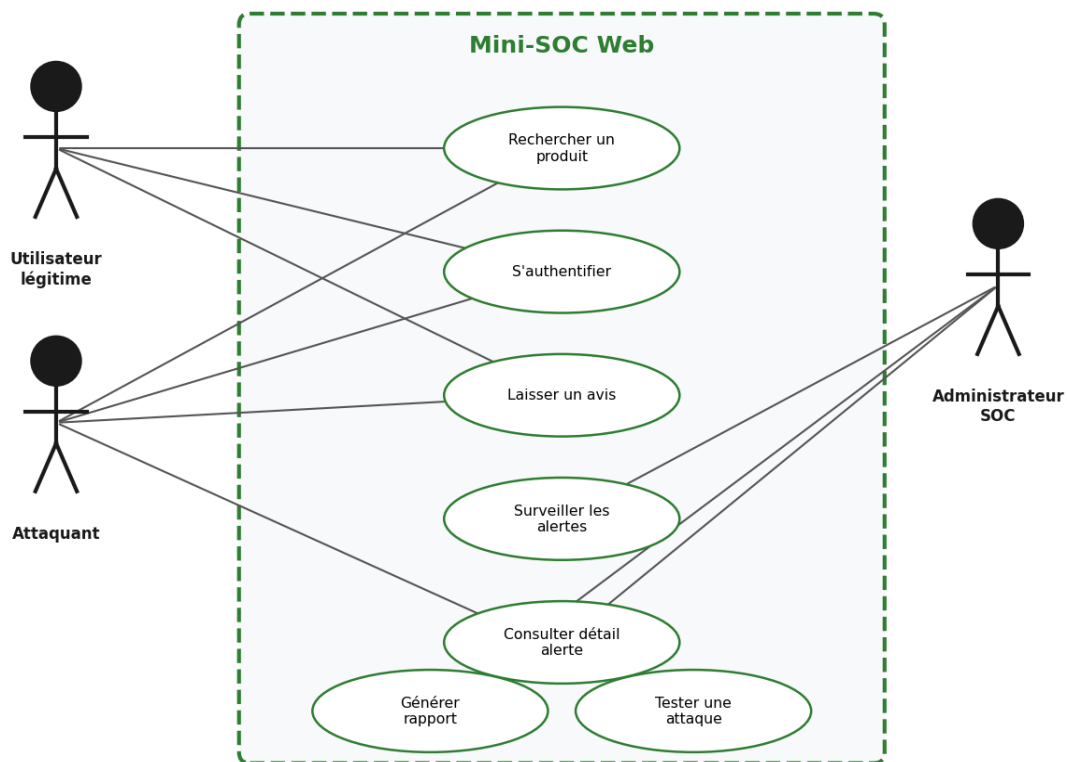


Figure 3.1 : Diagramme de cas d'utilisation — Acteurs et interactions avec le Mini-SOC Web

4.6 Diagramme de séquence

Le diagramme de séquence illustre le déroulement temporel d'une attaque de type injection SQL, depuis l'envoi du payload jusqu'à l'affichage de l'alerte dans le dashboard. Cette modélisation met en évidence les interactions entre les composants et la chronologie des événements.

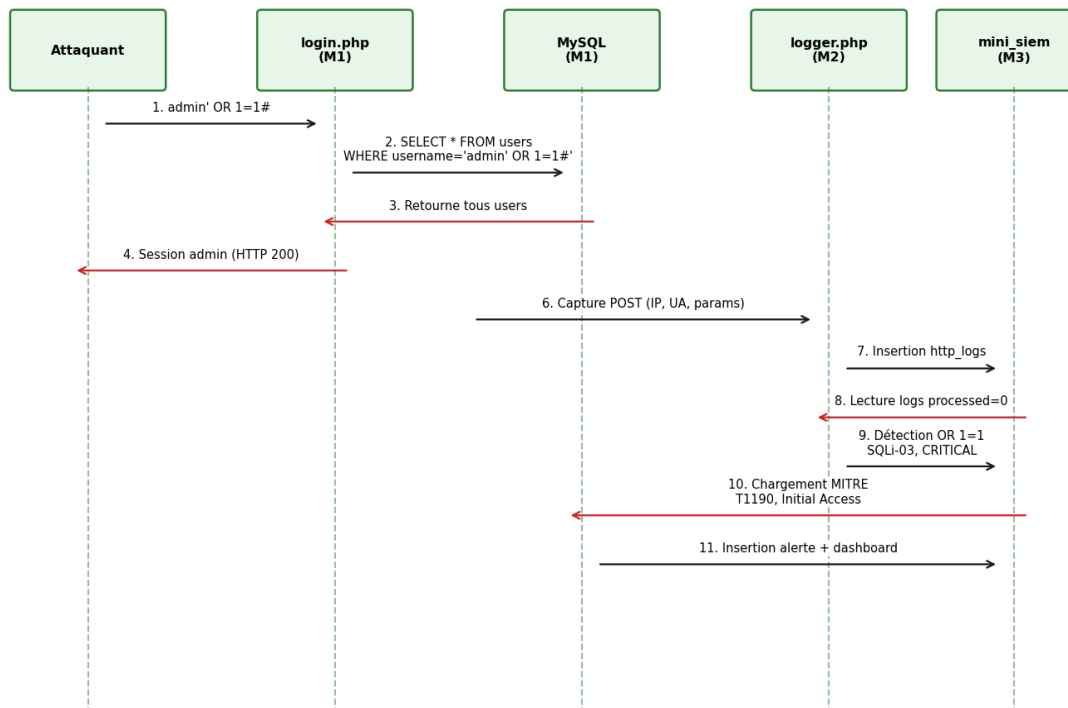


Figure 3.2 : Diagramme de séquence — Scénario d'injection SQL du payload à l'alerte MITRE

4.7 Cadre éthique et légal

4.7.1 Considérations éthiques

Le projet Mini-SOC Web est conçu dans un cadre strictement pédagogique et académique, dans le respect des principes déontologiques de la recherche en cybersécurité. L'application web vulnérable est déployée en environnement local (localhost uniquement) et ne doit en aucun cas être exposée sur un réseau public ou déployée en production.

Des avertissements éthiques clairs sont affichés sur les pages vulnérables de l'application, rappelant que les failles présentées sont intentionnelles et à des fins éducatives uniquement. Toutes les vulnérabilités sont documentées, analysées et accompagnées de contre-mesures, dans un objectif

de formation à la sécurité défensive.

Le projet respecte le code de déontologie de l'EMSI et les principes de responsabilité numérique. Aucune activité malveillante n'est encouragée, et l'ensemble du travail vise à former des professionnels de la cybersécurité capables de protéger les systèmes d'information contre les menaces réelles.

4.7.2 Cadre légal

Le projet est conforme à la **loi 09-08** marocaine relative à la protection des données à caractère personnel, qui impose le respect de la vie privée et la protection des informations personnelles. Aucune donnée réelle d'individu n'est collectée ni traitée dans le cadre de ce projet.

Le projet respecte également la **loi 31-03** sur la cybersécurité et la lutte contre la cybercriminalité, en ce qu'il ne vise pas à faciliter des activités illégales mais à former des professionnels de la cybersécurité. Les données utilisées dans l'application Atlas MiniShop sont entièrement fictives (comptes clients, commandes, paiements) et générées à des fins de démonstration.

Aucune donnée réelle n'est collectée ni traitée par le système de collecte de logs ni par le moteur de détection. Les adresses IP, les payloads et les événements enregistrés proviennent exclusivement de tests contrôlés effectués dans l'environnement local de développement.

4.7.3 Bonnes pratiques de sécurité

Isolation réseau : L'application vulnérable est accessible uniquement en localhost (127.0.0.1) et n'est pas exposée à Internet. Aucun service n'écoute sur une interface réseau externe.

Avertissements clairs : Chaque page vulnérable affiche un bandeau d'avertissement indiquant que la page contient des failles intentionnelles à des fins pédagogiques.

Documentation des risques : Chaque vulnérabilité est documentée avec sa description technique, son impact potentiel, les étapes de reproduction et les contre-mesures recommandées.

Sensibilisation : Le projet vise à sensibiliser les étudiants aux deux dimensions de la sécurité — offensive (pour comprendre l'attaque) et défensive (pour construire la protection) — dans une démarche éthique et responsable.

Chapitre 5 : Les Attaques Web

5.1 Injection SQL (SQLi)

5.1.1 Mécanismes et principes

L'injection SQL est une vulnérabilité de sécurité qui permet à un attaquant d'interférer avec les requêtes qu'une application adresse à sa base de données. Elle se produit lorsque des entrées utilisateur sont directement concaténées dans des requêtes SQL sans validation, sans échappement ni utilisation de requêtes préparées. L'attaquant peut alors injecter du code SQL arbitraire, modifiant la sémantique de la requête originale.

Le mécanisme fondamental repose sur la confusion entre données et instructions. Lorsque l'application construit une requête par simple concaténation de chaînes, le moteur SQL ne peut pas distinguer les données légitimes des instructions malveillantes. Une apostrophe non échappée, par exemple, ferme prématurément une chaîne de caractères dans la requête, permettant à l'attaquant d'ajouter des opérateurs logiques, des unions ou des commandes destructrices.

Dans le contexte d'Atlas MiniShop, l'injection SQL est présente sur plusieurs pages. La page login.php construit la requête d'authentification par concaténation directe : `SELECT * FROM users WHERE username = '$username' AND password = '$password'`. Un attaquant injectant `admin' OR 1=1#` modifie la condition logique, rendant la vérification du mot de passe inutile. La page search.php est vulnérable via la clause `LIKE '%$q%'`. La page product.php utilise l'identifiant de produit directement dans la requête sans cast ni préparation.

Le code suivant extrait de login.php montre la construction vulnérable de la requête SQL d'authentification :

```
$username = $_POST['username'] ?? '';
$password = $_POST['password'] ?? '';

// Vulnérabilité volontaire : SQLi par concaténation directe
$sql = "SELECT id, username, full_name, email, role
FROM users
WHERE username = '$username' AND password = '$password'
LIMIT 1";

$userRow = db_one($sql);
if ($userRow) {
    $_SESSION['user'] = $userRow;
```

```

http_response_code(200);
header('Location: account.php');
exit;
}
http_response_code(401);
  
```

5.1.2 Variantes et techniques d'injection

Plusieurs variantes d'injection SQL sont implémentées ou démontrables dans le projet :

- **Injection SQL classique (Boolean-based)** : Utilisation de l'opérateur OR 1=1 pour rendre une condition toujours vraie. Exemple : ' OR '1'=1 dans la barre de recherche retourne l'intégralité du catalogue.
- **Injection SQL par commentaire** : Utilisation de -- ou # pour ignorer le reste de la requête. Exemple : admin' OR 1=1# dans le login annule la vérification du mot de passe.
- **Injection SQL par UNION** : Utilisation de UNION SELECT pour extraire des données d'autres tables. Exemple : ' UNION SELECT username, password, email, role, created_at FROM users -- permet de récupérer les credentials.
- **Injection SQL basée sur le temps (Time-based)** : Utilisation de fonctions comme SLEEP(5) ou BENCHMARK() pour retarder la réponse et confirmer l'existence de la vulnérabilité.
- **Injection SQL dans l'INSERT** : Les pages checkout.php et review.php insèrent des données utilisateur sans échappement, permettant l'injection de valeurs malveillantes dans la base.

5.1.3 Impacts et risques

L'impact d'une injection SQL peut être extrêmement grave. Il inclut : le contournement de l'authentification et l'accès non autorisé aux comptes administrateurs ; l'exfiltration de données sensibles (clients, commandes, paiements) ; la modification ou la suppression de données ; l'exécution de commandes système via des fonctions étendues (xp_cmdshell) ; et dans les cas les plus critiques, la prise de contrôle complète du serveur de base de données. Dans notre projet, la démonstration se limite à la preuve de concept : contournement d'authentification, exfiltration de données par UNION et fuite d'informations par erreurs SQL.

5.1.4 Table de détection SQLi

ID Règle	Pattern / Condition	Gravité	Description
----------	---------------------	---------	-------------

SQLi-01	Apostrophe, guillemet, commentaire SQL	ÉLEVÉ	Détection de caractères d'échappement suspects dans les paramètres.
SQLi-02	UNION, SELECT, INSERT, DELETE, DROP	CRITIQUE	Détection de mots-clés de manipulation de données.
SQLi-03	OR/AND suivi d'une tautologie (1=1, 'a'='a')	ÉLEVÉ	Détection de conditions logiques toujours vraies.
SQLi-04	SLEEP(), BENCHMARK(), WAITFOR	CRITIQUE	Détection de fonctions de retard utilisées en blind SQLi.
SQLi-05	EXEC, xp_cmdshell, fonctions étendues	CRITIQUE	Détection de tentatives d'exécution de commandes système.

5.1.5 Contre-mesures

Les contre-mesures contre l'injection SQL sont bien connues et doivent être appliquées systématiquement. La mesure fondamentale est l'utilisation de **requêtes préparées (prepared statements)** avec paramètres liés, qui séparent clairement le code SQL des données. L'appel à `mysqli_prepare()` suivi de `mysqli_stmt_bind_param()` garantit que les entrées utilisateur sont traitées comme des données, jamais comme du code. L'échappement des entrées avec `mysqli_real_escape_string()` constitue une mesure de second niveau. La validation des types de données (cast en entier pour les identifiants) et l'utilisation de listes blanches pour les valeurs énumérées (statuts de commande) renforcent la défense. Enfin, le principe du moindre privilège pour le compte de base de données de l'application limite l'impact d'une injection réussie.

5.2 Cross-Site Scripting (XSS)

5.2.1 Mécanismes et principes

Le cross-site scripting (XSS) est une vulnérabilité qui permet à un attaquant d'injecter du code JavaScript malveillant dans des pages web visualisées par d'autres utilisateurs. Elle se produit lorsque l'application affiche des données utilisateur sans les échapper ni les nettoyer. Le navigateur de la victime exécute alors le script injecté dans le contexte de confiance du site, permettant le vol de cookies, le détournement de session, le défigement du site ou la redirection vers des sites malveillants.

Le mécanisme fondamental est l'absence de séparation entre le contenu actif (HTML, JavaScript) et le contenu passif (texte, données). Lorsque l'application insère une entrée utilisateur directement dans le DOM sans appliquer `htmlspecialchars()` ou des politiques de Content Security Policy, le navigateur interprète les balises et les gestionnaires d'événements comme du code légitime.

Dans Atlas MiniShop, le XSS est présent sous trois formes. Le **XSS réfléchi** se produit sur `search.php`, où le terme de recherche `$_GET['q']` est affiché sans échappement dans la page de résultats. Le **XSS stocké** se produit sur `product.php`, où les avis des clients sont affichés sans nettoyage. Le **XSS basé sur le DOM** se produit dans le fichier `shop.js`, où `window.location.hash` est injecté dans `innerHTML` via une manipulation du DOM.

4.2.2 Variantes d'XSS

- **XSS réfléchi (Reflected XSS)** : Le payload est inclus dans une requête HTTP et immédiatement réfléchi par le serveur dans la réponse. La victime doit cliquer sur un lien spécialement conçu. Exemple : `<script>alert('XSS')</script>` dans la barre de recherche.

- **XSS stocké (Stored XSS)** : Le payload est persistant dans la base de données et affiché à chaque consultation de la page. Plus dangereux car aucune interaction de la victime n'est requise au-delà de la navigation normale. Exemple : un avis contenant `<script>alert(document.cookie)</script>` est stocké et exécuté pour tous les visiteurs.

- **XSS basé sur le DOM (DOM-based XSS)** : Le payload modifie la structure du document côté client sans interaction avec le serveur. Exemple : la manipulation du hash de l'URL (`#promo=<script>...`) injecté dans `innerHTML` via JavaScript.

5.2.3 Impacts et risques

L'impact du XSS dépend du contexte d'exécution et des privilèges de la victime. Un XSS sur une page publique peut servir à défigurer le site, à voler les cookies de session (permettant le détournement de session), à enregistrer les frappes clavier (keylogging), à exécuter des actions au nom de l'utilisateur (CSRF via XSS) ou à rediriger les visiteurs vers des sites de phishing. Lorsque la victime est un administrateur, l'impact est maximal : l'attaquant peut créer un nouvel administrateur, modifier les paramètres du site ou accéder à des données sensibles. Dans notre projet, la démonstration se limite à l'exécution d'alertes et à la lecture du cookie de session, le serveur étant local et isolé.

5.2.4 Table de détection XSS

ID Règle	Pattern / Condition	Gravité	Description
XSS-01	<script> ou balises d'événement	CRITIQUE	Détection d'injection de balises script actives.
XSS-02	onerror, onload, onclick, onfocus	ÉLEVÉ	Détection de gestionnaires d'événements inline.
XSS-03	javascript: dans les URLs	CRITIQUE	Détection de protocoles de script dans les liens.
XSS-04	alert(), eval(), document.cookie	ÉLEVÉ	Détection de fonctions JavaScript dangereuses.
XSS-05	Encodages URL (%3C, %3E) et SVG	MOYEN	Détection de payloads encodés ou vecteurs SVG.

5.2.5 Contre-mesures

Les contre-mesures contre le XSS sont multiples et complémentaires. La première mesure est l'**encodage des sorties** : toute donnée utilisateur affichée dans une page doit être traitée par `htmlspecialchars()` avec le flag `ENT_QUOTES`, convertissant les caractères spéciaux en entités HTML. La deuxième mesure est la **validation des entrées** : les champs texte doivent être nettoyés avec des listes blanches de caractères autorisés. La troisième mesure est l'utilisation d'une **Content Security Policy (CSP)** qui restreint les sources de scripts exécutables. La quatrième mesure est l'activation du flag **HttpOnly** sur les cookies de session, empêchant leur accès via JavaScript. Enfin, le nettoyage des entrées côté client (JavaScript) ne doit jamais remplacer le nettoyage côté serveur, qui reste la ligne de défense principale.

4.3 Attaque par force brute (Brute Force)

5.3.1 Mécanismes et principes

L'attaque par force brute est une méthode d'attaque qui consiste à tester systématiquement un grand nombre de combinaisons de noms d'utilisateur et de mots de passe jusqu'à trouver la bonne. Elle s'appuie sur la persistance et la rapidité des machines, qui peuvent effectuer des milliers de tentatives par seconde. L'attaque peut être ciblée (un seul compte, plusieurs mots de passe) ou distribuée (plusieurs comptes, dictionnaire de mots de passe communs).

Dans le contexte d'Atlas MiniShop, la page `login.php` est vulnérable à la force brute car elle ne met en place aucune limitation du nombre de tentatives, aucun délai entre les échecs, aucun

CAPTCHA et aucun verrouillage de compte. L'attaquant peut soumettre des requêtes POST répétées avec des identifiants différents jusqu'à obtenir un code HTTP 200, indiquant une authentification réussie. La simplicité des mots de passe des comptes de démonstration (123456, password) facilite grandement cette attaque.

Le mécanisme de détection repose sur l'analyse comportementale. Le Mini-SIEM compte le nombre de requêtes POST sur login.php depuis une même adresse IP dans une fenêtre de soixante secondes. Si ce nombre dépasse le seuil de cinq, une alerte de type Brute Force est générée. Le système détecte également les séries de codes 401 (Unauthorized) consécutifs et les User-Agent suspects associés à des outils d'attaque automatique (Hydra, Medusa, sqlmap, curl).

4.3.2 Impacts et risques

L'impact d'une attaque par force brute réussie est l'accès non autorisé à un compte utilisateur. Si le compte compromis est un compte administrateur, l'attaquant obtient un contrôle total sur l'application. Même un compte client standard peut être utilisé pour accéder aux données personnelles, passer des commandes frauduleuses ou lancer des attaques internes. La compromission de comptes est souvent la première étape d'une attaque plus complexe (lateral movement, privilege escalation).

5.3.3 Table de détection Brute Force

ID Règle	Condition	Gravité	Description
BF-01	≥ 5 POST sur login.php, même IP, $< 60s$	ÉLEVÉ	Détection de tentatives rapides de connexion depuis une même source.
BF-02	≥ 3 codes 401 consécutifs, même IP	ÉLEVÉ	Détection d'échecs d'authentification répétés.
BF-03	User-Agent contenant Hydra, sqlmap, curl	MOYEN	Détection d'outils d'attaque automatique.
BF-04	≥ 10 usernames distincts, même IP, < 5 min	CRITIQUE	Détection d'attaque distribuée sur plusieurs comptes.
BF-05	Intervalle moyen < 500 ms sur ≥ 10 requêtes	MOYEN	Détection de rythme robotique incompatible avec un humain.

5.3.4 Contre-mesures

Les contre-mesures contre la force brute sont multiples. La mesure fondamentale est l'implémentation d'un **rate limiting** : après trois échecs consécutifs, le compte est verrouillé pour cinq minutes et l'IP est placée sur une liste grise. L'ajout d'un **CAPTCHA** après le premier échec empêche l'automatisation. L'utilisation d'un **hachage fort** (bcrypt, Argon2) avec salage rend les attaques offline impraticables. La **double authentification (2FA)** constitue une barrière supplémentaire. Enfin, la **détection comportementale** (heures de connexion inhabituelles, localisation géographique) permet d'identifier les tentatives suspectes avant qu'elles ne réussissent.

5.4 Référencement MITRE ATT&CK;

Le framework MITRE ATT&CK; est un référentiel de connaissances globalement reconnu qui modélise les tactiques et techniques utilisées par les attaquants dans le monde réel. Intégrer ce référentiel dans le Mini-SOC Web apporte une dimension professionnelle et académique, permettant de classer chaque attaque détectée selon une taxonomie standardisée. Cette classification facilite la communication entre les analystes, la génération de rapports normalisés et l'alignement avec les pratiques industrielles.

Attaque	ID MITRE	Tactique	Technique	OWASP 2021	Sévérité
Injection SQL	T1190	Initial Access	Exploit Public-Facing Application	A03 — Injection	CRITIQUE
Cross-Site Scripting	T1059.007	Execution	Command and Scripting Interpreter : JavaScript	A03 — Injection	ÉLEVÉ
Force Brute	T1110	Credential Access	Brute Force	A07 — Identification and Authentication Failures	ÉLEVÉ

Le mapping MITRE enrichit chaque alerte générée par le Mini-SIEM avec des métadonnées structurées : l'identifiant de technique, la tactique associée, la description détaillée, les indicateurs de compromission (IoC) observés, les réponses recommandées immédiates, les étapes d'investigation approfondie et les mesures correctives à long terme. Cette enrichment transforme une simple alerte technique en un incident de sécurité documenté, traçable et actionnable.

Chapitre 6 : Conception et Développement

6.1 Module M1 — Application Atlas MiniShop

6.1.1 Architecture de l'application

L'application Atlas MiniShop est structurée selon une architecture monolithique en trois couches, typique des applications PHP traditionnelles. La couche de présentation est constituée des fichiers PHP qui génèrent le HTML envoyé au navigateur. La couche de logique métier est assurée par les scripts PHP qui traitent les formulaires, gèrent les sessions et interagissent avec la base de données. La couche de données est représentée par la base MySQL qui stocke l'ensemble des informations du site.

Le répertoire principal contient les pages publiques : `index.php` (accueil), `search.php` (recherche), `product.php` (fiche produit), `cart.php` (panier), `checkout.php` (commande), `track_order.php` (suivi), `login.php` (authentification), `account.php` (compte client), `admin.php` (back-office) et `logout.php` (déconnexion). Le répertoire `includes/` contient les fichiers partagés : `header.php`, `footer.php`, `helpers.php`, `logger.php` et le répertoire `config/` contient `db.php` et `app.php`. Le répertoire `assets/` regroupe les ressources statiques : CSS, JavaScript et images. Le répertoire `database/` contient les scripts SQL de création des tables.

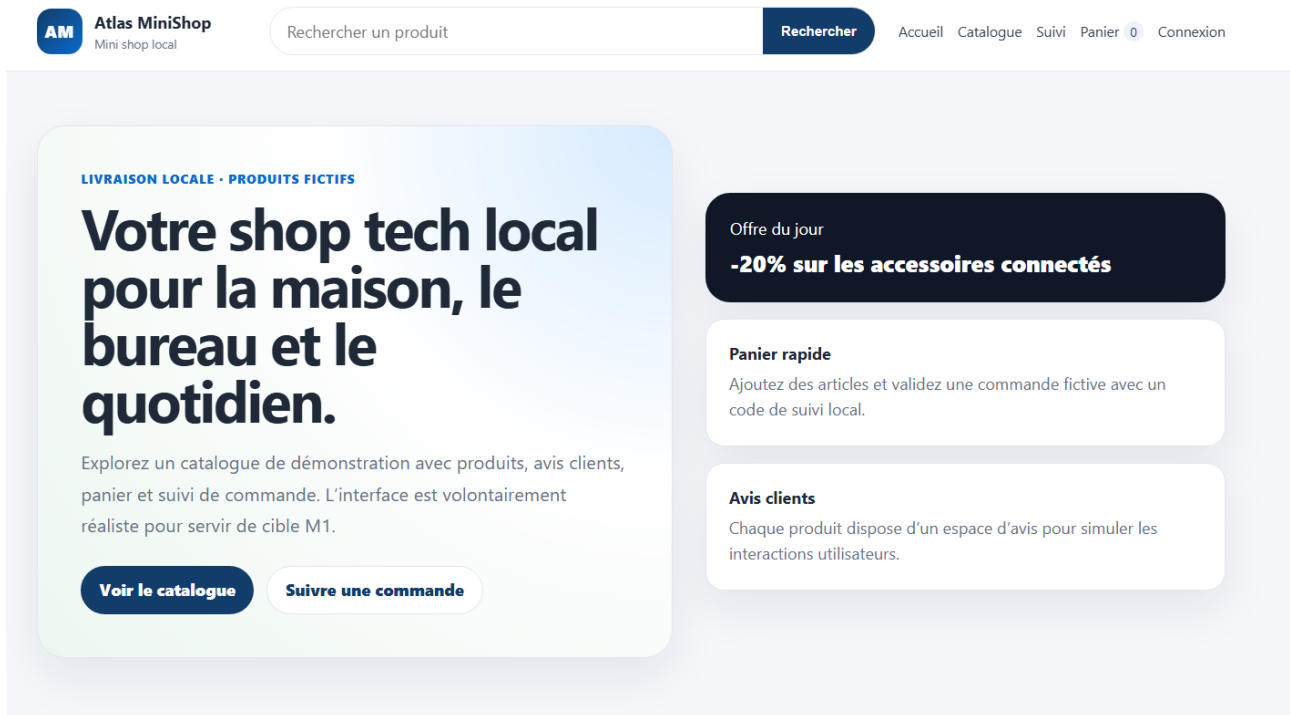


Figure 8.1 : Page d'accueil d'Atlas MiniShop — Catalogue des produits en vedette

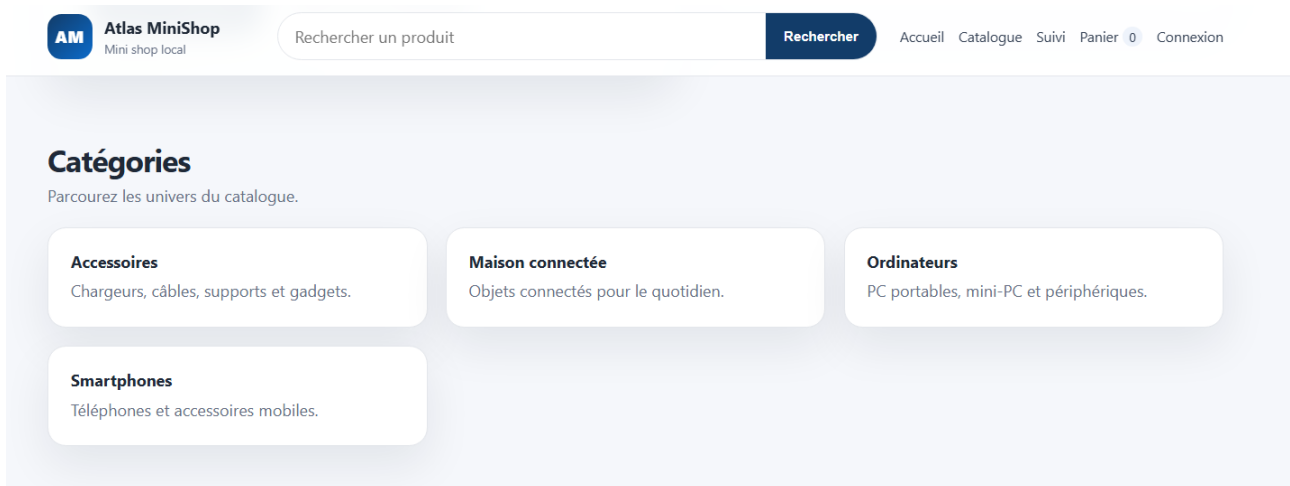


Figure 8.2 : Page Catégories d'Atlas MiniShop — Navigation par catégories de produits

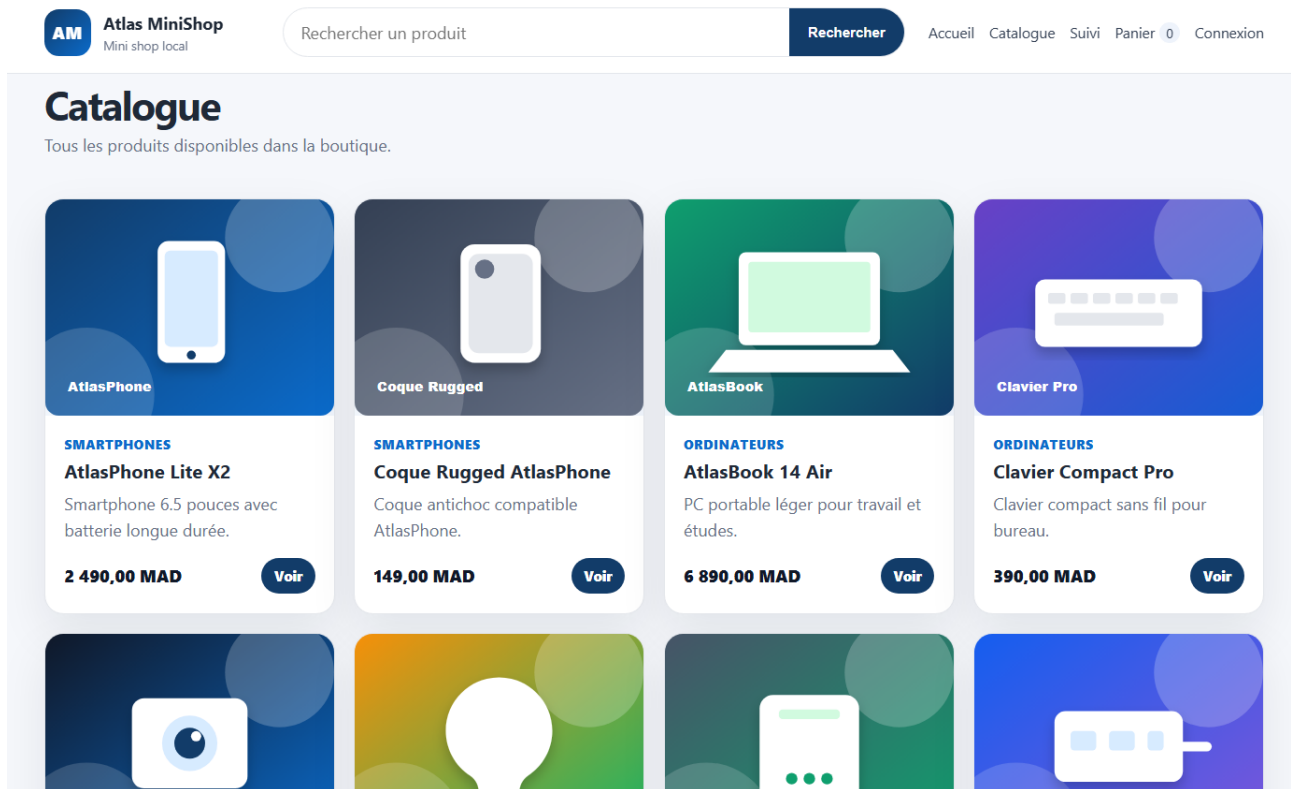


Figure 8.3 : Page Catalogue produits — Liste des produits avec filtres

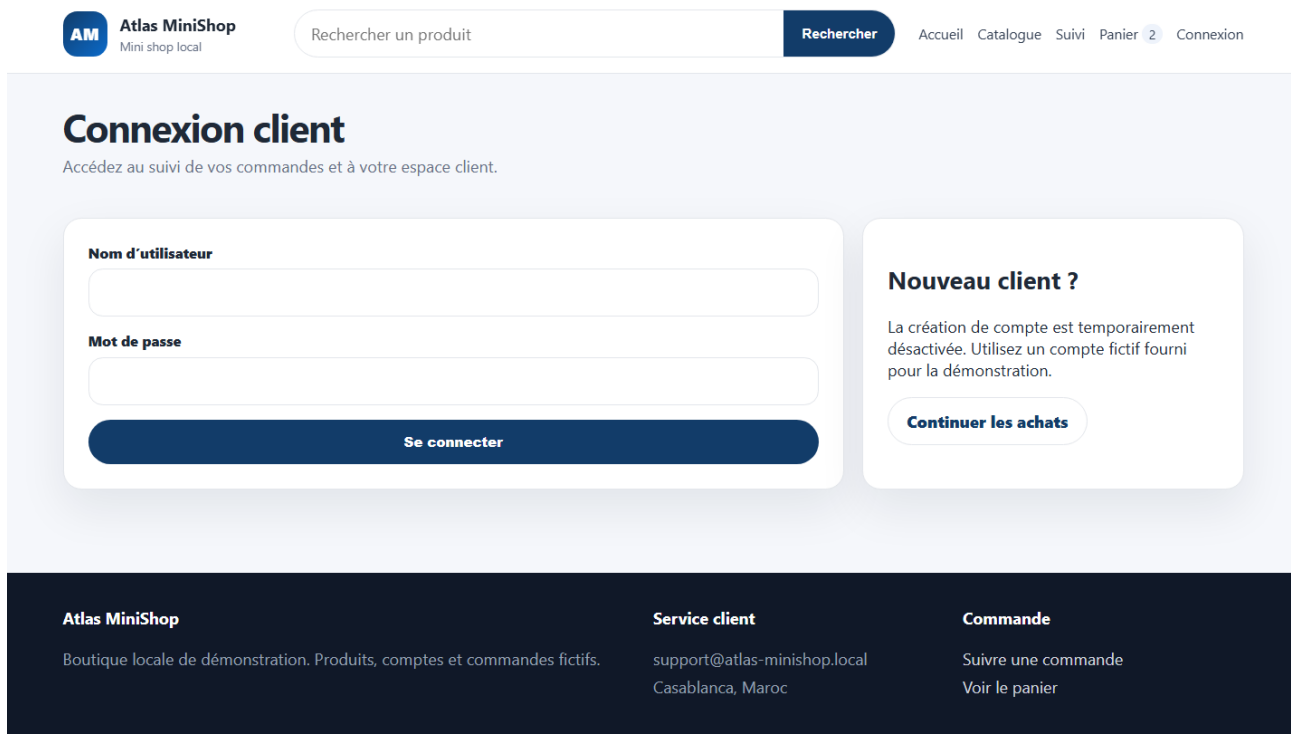


Figure 8.4 : Page de connexion — Formulaire d'authentification vulnérable

AM Atlas MiniShop
Mini shop local

Rechercher un produit **Rechercher** Accueil Catalogue Suivi Panier 2 Connexion

Connexion client

Accédez au suivi de vos commandes et à votre espace client.

Nom d'utilisateur
admin

Mot de passe
.....

Se connecter

Nouveau client ?
La création de compte est temporairement désactivée. Utilisez un compte fictif fourni pour la démonstration.

Continuer les achats

Atlas MiniShop
Boutique locale de démonstration. Produits, comptes et commandes fictifs.

Service client
support@atlas-minishop.local
Casablanca, Maroc

Commande
Suivre une commande
Voir le panier

Figure 8.5 : Page Connexion client — Espace client avec profil et historique

AM Atlas MiniShop
Mini shop local

Rechercher un produit **Rechercher** Accueil Catalogue Suivi Panier 0 Connexion

AtlasPhone Lite X2

Smartphones

Smartphone 6.5 pouces avec batterie longue durée.

Un téléphone fictif pensé pour les démonstrations de catalogue : grand écran, autonomie solide et design compact.

2 490,00 MAD ~~2 890,00 MAD~~

Stock : 18

Quantité
1

Ajouter au panier

Figure 8.6 : Page produit avec formulaire d'avis — Surface d'attaque XSS stocké

AM Atlas MiniShop
Mini shop local

Rechercher un produit

Rechercher

Accueil Catalogue Suivi Panier 0 Connexion

1

Ajouter au panier

Avis clients

2 avis sur ce produit.

Hamza ★★★★★
Design propre et livraison fictive rapide.
2026-05-15 14:43:42

Meryem ★★★★★
Très bon rapport qualité/prix pour une boutique de démonstration.
2026-05-13 14:43:42

Ajouter un avis

Nom

Note
5 étoiles

Commentaire

Publier

Figure 8.7 : Page produit avec avis — Détail d'un produit et avis clients

AM Atlas MiniShop
Mini shop local

Rechercher un produit

Rechercher

Accueil Catalogue Suivi Panier 2 Compte Admin Sortir

Back-office

Vue interne pour gérer les commandes fictives.

Commandes
2 commandes

Avis clients
4 avis

Comptes
5 utilisateurs

Commandes récentes

Filtre actuel : all

Tous

Filtrer

CODE	CLIENT	CONTACT	NOTES	TOTAL	STATUT	DATE
AMS-20250102-2002	Bob Client	bob@example.local 0600000002	Appeler avant livraison.	715,00 MAD	processing	2026-05-15 14:43:42
AMS-20250101-1001	Alice Client	alice@example.local 0600000001	Livraison matin.	2 515,00 MAD	delivered	2026-05-05 14:43:42

Figure 8.8 : Back-office administrateur — Gestion des commandes et avis

Atlas MiniShop
 Mini shop local

Rechercher un produit

Rechercher

[Accueil](#)
[Catalogue](#)
[Suivi](#)
[Panier 0](#)
[Compte](#)
[Sortir](#)

Mon compte

Bonjour Alice Client.

Profil

Utilisateur : alice

Email : alice@example.local

Rôle : customer

Mes commandes

CODE	TOTAL	STATUT	DATE
AMS-20250101-1001	2 515,00 MAD	delivered	2026-05-05 14:43:42

Atlas MiniShop
 Boutique locale de démonstration. Produits, comptes et commandes fictifs.

Service client
 support@atlas-minishop.local
 Casablanca, Maroc

Commande
[Suivre une commande](#)
[Voir le panier](#)

Figure 7.9 : Espace client — Profil et historique des commandes

6.1.2 Vulnérabilités intentionnelles

Les vulnérabilités sont introduites par l'absence délibérée de mesures de sécurité standard. Le fichier config/db.php utilise des requêtes construites par concaténation directe : toutes les requêtes passent par des fonctions db_query() et db_one() qui n'utilisent pas mysqli_prepare(). Le fichier includes/helpers.php définit une fonction e() comme alias de htmlspecialchars(), mais cette fonction n'est pas systématiquement appelée sur les données affichées. Le fichier config/app.php configure le cookie de session avec l'attribut httponly => false, permettant l'accès au cookie via JavaScript. Le fichier assets/js/shop.js utilise innerHTML pour injecter le contenu du hash de l'URL, créant une vulnérabilité DOM-based XSS. Ces choix sont documentés, isolés et utilisés exclusivement à des fins pédagogiques.

6.1.3 Fonctionnalités e-commerce

Malgré sa vocation pédagogique, Atlas MiniShop implémente des fonctionnalités e-commerce complètes et réalistes. Le catalogue présente des catégories (Accessoires, Maison, Ordinateurs, Smartphones) avec des fiches produits détaillées (nom, description, prix, stock, image). Le système de panier gère l'ajout, la suppression et la modification des quantités. Le passage de commande collecte les informations client et génère un code de suivi unique. L'espace client permet de consulter l'historique des commandes et leur statut (pending, processing, shipped, delivered). Le back-office administrateur offre une vue synthétique des statistiques (commandes, avis, comptes),

un filtre par statut de commande et une table de gestion. Ces fonctionnalités rendent l'application crédible et fournissent un contexte réaliste pour les attaques.

AM Atlas MiniShop
Mini shop local

Rechercher un produit

Rechercher

Accueil Catalogue Suivi Panier 0 Connexion

Suivi de commande

Entrez votre code de commande pour afficher son état.

Code de commande

AMS-20250101-1234

Rechercher

Aucune recherche lancée.

Atlas MiniShop
Boutique locale de démonstration. Produits, comptes et commandes fictifs.

Service client
support@atlas-minishop.local
Casablanca, Maroc

Commande
Suivre une commande
Voir le panier

Figure 8.10 : Page Suivi de commande — Formulaire de suivi par code de commande

AM Atlas MiniShop
Mini shop local

Rechercher un produit

Rechercher

Accueil Catalogue Suivi Panier 0 Connexion

Suivi de commande

Entrez votre code de commande pour afficher son état.

Code de commande

AMS-20250101-1001

Rechercher

Commande AMS-20250101-1001

delivered · 2026-05-05 14:43:42

PRODUIT	QTÉ	PRIX
AtlasPhone Lite X2	1	2 490,00 MAD

Total : 2 515,00 MAD

Adresse :
12 Rue Demo, Casablanca

Atlas MiniShop
Boutique locale de démonstration. Produits, comptes et commandes fictifs.

Service client
support@atlas-minishop.local

Commande
Suivre une commande

Figure 8.11 : Résultat suivi commande — Détail de la commande et statut de livraison

Atlas MiniShop
 Mini shop local

Rechercher un produit

Rechercher

[Accueil](#)
[Catalogue](#)
[Suivi](#)
[Panier 0](#)
[Connexion](#)

Panier

Vérifiez vos produits avant commande.

Votre panier est vide.
 [Voir le catalogue](#)

Résumé

Sous-total	0,00 MAD
Livraison	0,00 MAD
Total	0,00 MAD

Atlas MiniShop
 Boutique locale de démonstration. Produits, comptes et commandes fictifs.

Service client
 support@atlas-minishop.local
 Casablanca, Maroc

Commande
[Suivre une commande](#)
[Voir le panier](#)

Figure 8.12 : Page Panier — Récapitulatif des articles avant commande

6.2 Module M2 — Collecteur de logs HTTP

6.2.1 Principe de fonctionnement

Le collecteur de logs est implémenté dans le fichier `includes/logger.php` sous la forme d'une fonction `log_http_request()` appelée au début de chaque page via `include`. La fonction utilise `register_shutdown_function` pour s'assurer que la capture a lieu après le traitement complet de la requête, quelle que soit l'issue (succès, erreur, redirection). Cette approche garantit que le code de réponse HTTP final est correctement enregistré.

Pour chaque requête, les informations suivantes sont extraites : l'adresse IP source (priorité à `X-Forwarded-For` pour la compatibilité proxy, fallback sur `REMOTE_ADDR`), la méthode HTTP (GET ou POST), l'URL complète avec query string, les paramètres GET et POST sérialisés en JSON, la chaîne User-Agent, le code de réponse HTTP et le timestamp avec microsecondes. Le champ `processed` est initialisé à 0, indiquant que le log n'a pas encore été analysé par le Mini-SIEM.

Les pages surveillées incluent : `login.php` (authentification), `search.php` (recherche), `product.php` (fiche produit et avis), `review.php` (traitement POST des avis), `checkout.php` (passage de commande), `track_order.php` (suivi de commande) et `admin.php` (back-office). Chaque page appelle explicitement `log_http_request()` après l'initialisation de la connexion à la base de données.

Le code source suivant présente la fonction `log_http_request()` qui capture les requêtes HTTP via `register_shutdown_function` :

```
function log_http_request(): void
{
    register_shutdown_function(function () {
        global $conn;
        $ip = $_SERVER['REMOTE_ADDR'] ?? '0.0.0.0';
        $method = $_SERVER['REQUEST_METHOD'] ?? 'GET';
        $url = $_SERVER['PHP_SELF'] ?? '/';
        $allParams = [];
        foreach ($_GET as $k => $v) $allParams['GET_' . $k] = (string)$v;
        foreach ($_POST as $k => $v) $allParams['POST_' . $k] = (string)$v;
        $params = json_encode($allParams);
        $userAgent = $_SERVER['HTTP_USER_AGENT'] ?? '';
        $responseCode = http_response_code() ?: 200;
        $ip = mysqli_real_escape_string($conn, $ip);
        $params = mysqli_real_escape_string($conn, $params);
        mysqli_query($conn,
            "INSERT INTO http_logs
            (timestamp, ip_source, method, url, params, user_agent, response_code,
            processed)
            VALUES (NOW(), '$ip', '$method', '$url', '$params', '$userAgent',
            $responseCode, 0)"
        );
    });
}
```

6.2.2 Structure de la table `http_logs`

La table `http_logs` est définie dans le script `database/schema.sql`. Elle comporte les champs suivants : `id` (clé primaire auto-incrémentée), `timestamp` (datetime avec microsecondes), `ip_source` (varchar 45 pour compatibilité IPv6), `method` (varchar 10), `url` (varchar 500), `params` (text, contenant le JSON des paramètres), `user_agent` (varchar 500), `response_code` (int) et `processed` (tinyint, 0 ou 1). Un index est créé sur le champ `ip_source` pour accélérer les requêtes de comptage utilisées par les règles de détection de force brute. La contrainte d'unicité sur `log_id`, `attack_type` et `source_ip` dans la table `alerts` évite la duplication des alertes.

Table	Action	Lignes	Type	Interclassement	Taille	Perte
alerts	Parcourir Structure Rechercher Insérer Vider Supprimer	1	InnoDB	utf8mb4_unicode_ci	32,0 kio	-
categories	Parcourir Structure Rechercher Insérer Vider Supprimer	4	InnoDB	utf8mb4_general_ci	16,0 kio	-
http_logs	Parcourir Structure Rechercher Insérer Vider Supprimer	24	InnoDB	utf8mb4_unicode_ci	16,0 kio	-
orders	Parcourir Structure Rechercher Insérer Vider Supprimer	2	InnoDB	utf8mb4_general_ci	48,0 kio	-
order_items	Parcourir Structure Rechercher Insérer Vider Supprimer	2	InnoDB	utf8mb4_general_ci	48,0 kio	-
products	Parcourir Structure Rechercher Insérer Vider Supprimer	8	InnoDB	utf8mb4_general_ci	32,0 kio	-
reviews	Parcourir Structure Rechercher Insérer Vider Supprimer	5	InnoDB	utf8mb4_general_ci	32,0 kio	-
soc_alert_reads	Parcourir Structure Rechercher Insérer Vider Supprimer	5	InnoDB	utf8mb4_unicode_ci	16,0 kio	-
soc_login_attempts	Parcourir Structure Rechercher Insérer Vider Supprimer	0	InnoDB	utf8mb4_unicode_ci	16,0 kio	-
threat_score	Parcourir Structure Rechercher Insérer Vider Supprimer	12	InnoDB	utf8mb4_unicode_ci	16,0 kio	-
users	Parcourir Structure Rechercher Insérer Vider Supprimer	5	InnoDB	utf8mb4_general_ci	32,0 kio	-
11 tables	Somme	68	InnoDB	utf8mb4_general_ci	304,0 kio	0 o

Figure 8.13 : Structure de la base de données minisoc_shop sous phpMyAdmin

	id	timestamp	ip_source	method	uri	params	user_agent	response_code	processed
☐ Éditer ☐ Copier ☐ Supprimer	15	2026-05-21 13:09:07	1	GET	/Mini_soc_complet/search.php	["GET_q","h"]	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit...	200	1
☐ Éditer ☐ Copier ☐ Supprimer	16	2026-05-21 13:09:21	1	GET	/Mini_soc_complet/search.php	["GET_q"," OR '1'='1"]	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit...	200	1
☐ Éditer ☐ Copier ☐ Supprimer	17	2026-05-21 13:10:28	1	GET	/Mini_soc_complet/login.php	[]	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit...	200	1
☐ Éditer ☐ Copier ☐ Supprimer	18	2026-06-13 14:01:49	1	GET	/Mini_soc_complet/search.php	[]	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit...	200	0
☐ Éditer ☐ Copier ☐ Supprimer	19	2026-06-13 14:05:46	1	GET	/Mini_soc_complet/search.php	[]	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit...	200	0
☐ Éditer ☐ Copier ☐ Supprimer	20	2026-06-13 14:05:53	1	GET	/Mini_soc_complet/product.php	["GET_id","1"]	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit...	200	0

Figure 8.14 : Table http_logs — Logs des requêtes HTTP collectées

6.2.3 API REST du dashboard

L'API REST est implémentée dans le fichier soc/api.php. Elle fournit les endpoints suivants : GET /soc/api.php?action=dashboard retourne un objet JSON contenant les statistiques globales (total_alertes, sql_count, xss_count, bf_count, score_global, top_ips, alertes_récentes) ; POST /soc/api.php met à jour le statut d'une alerte (nouveau, en_cours, résolu, faux_positif) ; GET /soc/api.php?action=ensure_siem vérifie si le processus mini_siem.py est actif et le relance si nécessaire. L'API utilise des requêtes préparées mysqli pour prévenir les injections SQL sur le dashboard SOC.

6.3 Module M3 — Mini-SIEM Python

6.3.1 Architecture du moteur de détection

Le Mini-SIEM est implémenté dans le fichier `mini_siem/mini_siem.py`. Il s'agit d'un script Python autonome qui s'exécute en boucle infinie avec un intervalle de dix secondes entre chaque itération. Le script utilise la bibliothèque `mysql.connector` pour se connecter à la base de données MySQL, la bibliothèque `re` pour les expressions régulières, la bibliothèque `json` pour le chargement du mapping MITRE et la bibliothèque `logging` pour la traçabilité des opérations.

Le cycle de vie d'une itération est le suivant : connexion à MySQL, récupération des logs avec `processed = 0`, itération sur chaque log, application des règles de détection SQLi, XSS et Brute Force, vérification des doublons, insertion des alertes dans la table `alerts`, mise à jour du score de menace pour l'IP source, marquage du log comme traité (`processed = 1`), écriture dans le fichier de log `siem.log` et attente de dix secondes.

La boucle infinie constitue le cœur opérationnel du Mini-SIEM, exécutant un cycle d'analyse toutes les 10 secondes :

```
log("Mini-SIEM démarré – analyse toutes les 10 secondes")
while True:
    try:
        conn = connecter()
        cursor = conn.cursor(dictionary=True)
        analyser_logs(cursor, conn)
        cursor.close()
        conn.close()
    except mysql.connector.Error as e:
        log(f"[ERREUR MySQL] {e}")
    except Exception as e:
        log(f"[ERREUR] {e}")
    time.sleep(10)
```

6.3.2 Règles de détection

Les règles de détection sont définies comme une liste de dictionnaires Python, chacun contenant un identifiant, un type d'attaque, une expression régulière, une description, une condition comportementale optionnelle et un niveau de sévérité. Pour l'injection SQL, les règles couvrent les caractères d'échappement (`'`, `"`, `--`, `#`), les mots-clés de manipulation (`UNION`, `SELECT`, `INSERT`, `DELETE`, `DROP`, `UPDATE`), les tautologies logiques (`OR 1=1`, `AND 'a'='a'`), les fonctions de temporisation (`SLEEP()`, `BENCHMARK()`) et les fonctions étendues (`xp_cmdshell`). Pour le XSS, les règles détectent les balises `<script>`, les gestionnaires d'événements (`onerror`, `onload`, `onclick`,

onfocus), les protocoles javascript:, les appels à alert() et eval() et les encodages URL (%3C, %3E). Pour la force brute, les règles comptent les requêtes POST sur login.php par IP et par fenêtre temporelle, détectent les séries de codes 401 et identifient les User-Agent suspects.

Le code suivant présente les trois fonctions de détection du Mini-SIEM, basées sur les regex du mapping MITRE :

```
def detecter_sqli(texte):
    """Détection d'une SQLi via les regex du mapping MITRE"""
    for pattern in attacks["SQLi"]["detection_logic"]["regex_patterns"]:
        if re.search(pattern, texte, re.IGNORECASE):
            return True
    return False

def detecter_xss(texte):
    """Détection d'un XSS via les regex du mapping MITRE"""
    for pattern in attacks["XSS"]["detection_logic"]["regex_patterns"]:
        if re.search(pattern, texte, re.IGNORECASE):
            return True
    return False

def detecter_brute_force(ip, cursor):
    """Compte les échecs de connexion selon les seuils MITRE"""
    ips = variantes_ip(ip)
    placeholders = ", ".join(["%s"] * len(ips))
    cursor.execute(f"""
    SELECT COUNT(*) FROM http_logs
    WHERE ip_source IN ({placeholders})
    AND url LIKE '%login.php%'
    AND UPPER(method) = 'POST'
    AND response_code IN (200, 401, 403)
    AND timestamp >= NOW() - INTERVAL %s SECOND
    """, tuple(ips) + (BF_FENETRE,))
    result = cursor.fetchone()
    nb = int(result["nb"] or 0)
    return nb >= BF_SEUIL, nb
```

La fonction creer_alerte() enrichit chaque alerte avec les métadonnées MITRE et prévient les doublons :

```
def creer_alerte(log_entry, attack_key, payload, cursor, conn):
    info = attacks[attack_key]
    ip = normaliser_ip(log_entry.get("ip_source", ""))
```

```
# Anti-doublon : vérifie log_id + attack_type + source_ip
cursor.execute("""
SELECT id FROM alerts
WHERE log_id = %s AND attack_type = %s AND source_ip = %s
""", (log_entry["id"], info["attack_name"], ip))
if cursor.fetchone():
    return

reponse = info["recommended_response"]["immediate_actions"][0]
cursor.execute("""
INSERT INTO alerts (timestamp, log_id, attack_type, severity,
description, payload, source_ip, url, mitre_id, mitre_tactic,
mitre_technique, recommended_response, status)
VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s)
""", (datetime.now(), log_entry["id"], info["attack_name"],
info["severity"], f"{info['attack_name']} détectée",
payload[:500], ip, log_entry["url"], info["mitre_id"],
info["tactic"], info["technique"], reponse, "nouveau"))
conn.commit()
```

6.3.3 Système de scoring des menaces

Le système de scoring attribue des points à chaque IP attaquante en fonction des alertes générées. Une alerte SQLi CRITICAL apporte 15 points, une alerte XSS HIGH apporte 10 points et une alerte Brute Force HIGH apporte 8 points. Si une même IP génère trois alertes ou plus, un multiplicateur de 2 est appliqué. Le score total est classé selon quatre niveaux : LOW (0-20), MEDIUM (21-50), HIGH (51-100) et CRITICAL (>100). Ce score est affiché dans le dashboard sous forme de jauge colorée et utilisé pour le tri des alertes.

6.3.4 Enrichissement MITRE

L'enrichissement MITRE est réalisé par le chargement du fichier mitre_mapping.json au démarrage du Mini-SIEM. Pour chaque alerte détectée, le moteur identifie le type d'attaque, charge les informations MITRE correspondantes et les inclut dans l'alerte. Les champs enrichis sont : mitre_id (ex : T1190), mitre_tactic (ex : Initial Access), mitre_technique (ex : Exploit Public-Facing Application), recommended_response (ex : Vérifier la requête, corriger la faille SQLi et surveiller l'IP source) et severity (ex : CRITICAL). Cette enrichment transforme une alerte brute en un incident de sécurité documenté et actionnable.

		id	timestamp	log_id	attack_type	severity	description	payload	source_ip	url	mitre_id	mitre_tactic
☐	Éditer Copier Supprimer	9	2026-05-21 13:09:28	16	SQL Injection	CRITICAL	SQL Injection détectée sur /Mini_soc_complet/search.php	'OR '1'='1	127.0.0.1	/Mini_soc_complet/search.php	T1190	Initial Access
☐	Éditer Copier Supprimer	10	2026-06-13 15:08:03	39	SQL Injection	CRITICAL	SQL Injection détectée sur /Mini_soc_complet/search.php	'OR '1'='1	127.0.0.1	/Mini_soc_complet/search.php	T1190	Initial Access
☐	Éditer Copier Supprimer	11	2026-06-13 15:08:03	40	Cross-Site Scripting	HIGH	Cross-Site Scripting détectée sur /Mini_soc_complet/search.php	<script>alert("XSS")</script>	127.0.0.1	/Mini_soc_complet/search.php	T1059.007	Execution
☐	Éditer Copier Supprimer	12	2026-06-13 15:08:03	43	Cross-Site Scripting	HIGH	Cross-Site Scripting détectée sur /Mini_soc_complet/review.php	2 testeur 5	127.0.0.1	/Mini_soc_complet/review.php	T1059.007	Execution
☐	Éditer Copier Supprimer	13	2026-06-13 15:08:03	46	Cross-Site Scripting	HIGH	Cross-Site Scripting détectée sur /Mini_soc_complet/review.php	<script>alert("bonjour")</script>	127.0.0.1	/Mini_soc_complet/review.php	T1059.007	Execution
☐	Éditer Copier Supprimer	14	2026-06-13 15:08:03	49	SQL Injection	CRITICAL	SQL Injection détectée sur /Mini_soc_complet/login.php	admin'OR '1'='1 FSDSDFS	127.0.0.1	/Mini_soc_complet/login.php	T1190	Initial Access
☐	Éditer Copier Supprimer	15	2026-06-13 15:08:03	50	SQL Injection	CRITICAL	SQL Injection détectée sur /Mini_soc_complet/login.php	admin'OR '1'='1 FSDSDFS	127.0.0.1	/Mini_soc_complet/login.php	T1190	Initial Access
☐	Éditer Copier Supprimer	16	2026-06-13 15:08:03	52	SQL Injection	CRITICAL	SQL Injection détectée sur /Mini_soc_complet/login.php	admin'OR '1'='1 -- jasdghd	127.0.0.1	/Mini_soc_complet/login.php	T1190	Initial Access

Figure 8.15 : Table alerts — Alertes enrichies avec informations MITRE

		id	timestamp	log_id	attack_type	severity	description	payload	source_ip	url	mitre_id	mitre_tactic
☐	Éditer Copier Supprimer	12	2026-06-13 15:08:03	43	Cross-Site Scripting	HIGH	détectée sur /Mini_soc_complet/review.php	<script>alert("XSS")</script>	127.0.0.1	/Mini_soc_complet/review.php	T1059.007	Execution
☐	Éditer Copier Supprimer	13	2026-06-13 15:08:03	46	Cross-Site Scripting	HIGH	Cross-Site Scripting détectée sur /Mini_soc_complet/review.php	2 testeur 5	127.0.0.1	/Mini_soc_complet/review.php	T1059.007	Execution
☐	Éditer Copier Supprimer	14	2026-06-13 15:08:03	49	SQL Injection	CRITICAL	SQL Injection détectée sur /Mini_soc_complet/login.php	admin'OR '1'='1 FSDSDFS	127.0.0.1	/Mini_soc_complet/login.php	T1190	Initial Access
☐	Éditer Copier Supprimer	15	2026-06-13 15:08:03	50	SQL Injection	CRITICAL	SQL Injection détectée sur /Mini_soc_complet/login.php	admin'OR '1'='1 FSDSDFS	127.0.0.1	/Mini_soc_complet/login.php	T1190	Initial Access
☐	Éditer Copier Supprimer	16	2026-06-13 15:08:03	52	SQL Injection	CRITICAL	SQL Injection détectée sur /Mini_soc_complet/login.php	admin'OR '1'='1 -- jasdghd	127.0.0.1	/Mini_soc_complet/login.php	T1190	Initial Access
☐	Éditer Copier Supprimer	17	2026-06-13 15:08:03	53	SQL Injection	CRITICAL	SQL Injection détectée sur /Mini_soc_complet/login.php	admin'OR '1'='1 sfsfshg	127.0.0.1	/Mini_soc_complet/login.php	T1190	Initial Access
☐	Éditer Copier Supprimer	18	2026-06-13 15:08:03	55	SQL Injection	CRITICAL	SQL Injection détectée sur /Mini_soc_complet/login.php	admin'OR '1'='1 js	127.0.0.1	/Mini_soc_complet/login.php	T1190	Initial Access

Figure 8.16 : Table alerts détaillée phpMyAdmin — Vue complète des alertes avec MITRE

6.4 Module M4 — Dashboard SOC

6.4.1 Architecture du dashboard

Le dashboard SOC est une application web complète, indépendante de l'application e-commerce, située dans le répertoire soc/. Son architecture repose sur un noyau central (bootstrap.php) qui assure la sécurité, la connexion à la base de données et les fonctions partagées, et sur des pages spécialisées pour chaque fonctionnalité. Le noyau bootstrap.php (1986 lignes) implémente la gestion des sessions sécurisées (CSRF token, fingerprinting, rotation périodique, timeout), l'authentification avec hachage bcrypt, la limitation des tentatives de connexion (5 échecs entraînent un verrouillage de 5 minutes) et les en-têtes de sécurité (CSP, X-Frame-Options, X-Content-Type-Options).

Les pages du dashboard sont : index.php (vue d'ensemble), alerts.php (historique complet), detail.php (détail d'une alerte), api.php (API REST interne), settings.php (configuration), report.php (génération de rapports PDF), mitre.php (documentation MITRE) et guide.php (guide d'utilisation). Les ressources statiques sont regroupées dans soc/assets/soc.css (styles, thème clair/sombre, responsive design) et soc/assets/soc.js (rafraîchissement automatique, gestion des filtres, localStorage).

6.4.2 Vue d'ensemble (index.php)

La page d'accueil du dashboard affiche une vue d'ensemble temps réel des alertes de sécurité. Elle est composée de cartes statistiques : le nombre total d'alertes, le nombre d'alertes SQLi, le nombre d'alertes XSS, le nombre d'alertes Brute Force et le score global de menace. Un graphique de tendance temporelle montre l'évolution du nombre d'alertes sur les sept derniers jours. Un diagramme circulaire (donut) présente la répartition par niveau de gravité. Un tableau affiche le top 5 des IP les plus actives avec leur score de menace. Enfin, un tableau des alertes récentes présente les dix dernières alertes avec leur type, gravité, date et statut. Un clic sur une alerte ouvre le panneau de détail avec toutes les informations MITRE.

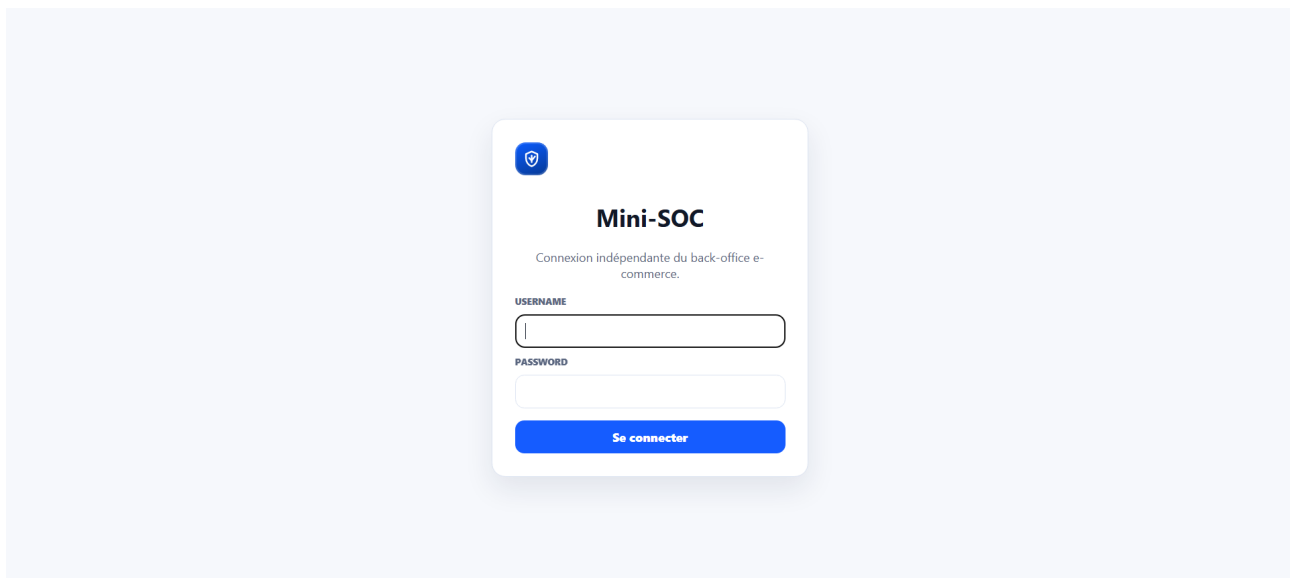


Figure 8.17 : Page de connexion du dashboard SOC — Authentification sécurisée

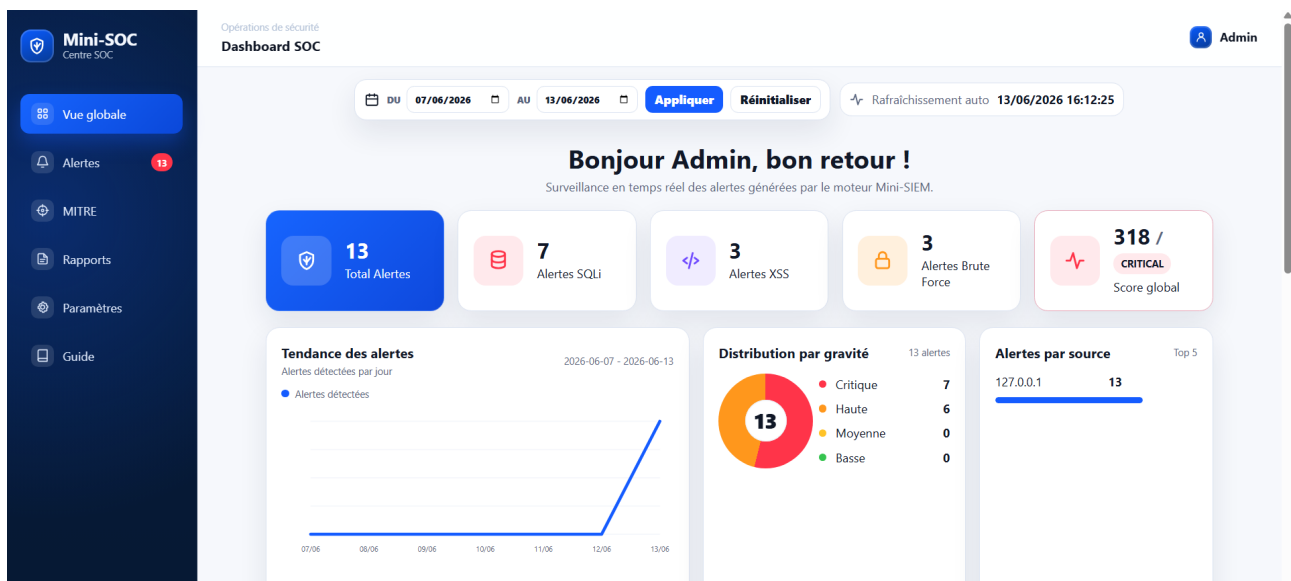


Figure 8.18 : Vue globale du dashboard SOC — Statistiques et alertes en temps réel

6.4.3 Historique et filtres (alerts.php)

La page d'historique affiche l'ensemble des alertes sous forme de tableau paginé et filtrable. Les filtres disponibles sont : type d'attaque (SQLi, XSS, Brute Force), niveau de gravité (CRITICAL, HIGH, MEDIUM, LOW), statut (Nouveau, En cours, Résolu, Faux positif) et période temporelle. L'administrateur peut marquer une ou plusieurs alertes comme lues, changer leur statut et ajouter des notes internes. Les filtres sont appliqués côté serveur via des requêtes préparées pour garantir la sécurité du dashboard lui-même.

6.4.4 Génération de rapports PDF (report.php)

Le module de génération de rapports produit des documents PDF natifs sans dépendance externe. Le script report.php construit le document PDF en format 1.4 inline, avec des objets PDF manuellement assemblés. Le rapport contient : une page de couverture avec le titre et la date, un résumé exécutif avec les statistiques globales, une analyse détaillée par type d'attaque avec les recommandations MITRE, une timeline des incidents et un glossaire des termes. Les paramètres du rapport (période, types d'attaque, gravité) sont configurables via l'interface web. Le fichier soc/report_recommendations.json fournit les contenus multilingues (français et anglais) des recommandations.

6.4.5 Guide d'utilisation (guide.php)

Le guide d'utilisation est intégré au dashboard et accessible sans authentification. Il présente le processus de gestion des incidents en quatre étapes : (1) Préparer le laboratoire — lancer

l'application vulnérable et le Mini-SIEM ; (2) Activer la détection — vérifier que le collecteur de logs fonctionne et que le Mini-SIEM est en cours d'exécution ; (3) Lire le dashboard — interpréter les alertes, comprendre les informations MITRE et évaluer la gravité ; (4) Traiter et documenter — appliquer les recommandations, changer le statut de l'alerte et générer un rapport. Ce guide constitue une documentation opérationnelle complète pour les utilisateurs du Mini-SOC.

5.5 Module M5 — Mapping MITRE ATT&CK;

5.5.1 Structure du fichier mitre_mapping.json

Le fichier mitre_mapping.json est un document JSON structuré qui définit la base de connaissances MITRE du projet. Pour chaque type d'attaque (SQLi, XSS, Brute Force), il contient : la description générale, les tactiques MITRE associées, les techniques avec leurs identifiants, les sous-techniques, les indicateurs de compromission (IoC) sous forme de listes de patterns et de User-Agent suspects, les regex de détection, les seuils de déclenchement, les réponses immédiates recommandées, les étapes d'investigation approfondie, les mesures correctives à long terme, les faux positifs possibles et les matériels de référence. La matrice de sévérité définit les niveaux CRITICAL, HIGH, MEDIUM et LOW avec leurs seuils de score et leurs délais de réponse (SLA).

5.5.2 Exemple de mapping pour l'injection SQL

Pour l'injection SQL, le mapping MITRE spécifie : l'identifiant T1190 (Exploit Public-Facing Application), la tactique Initial Access, la technique d'exploitation des applications web, les IoC (apostrophes non échappées, mots-clés UNION SELECT, fonctions SLEEP), les réponses immédiates (vérifier la requête dans les logs, corriger la faille avec des requêtes préparées, surveiller l'IP source), les étapes d'investigation (identifier les données potentiellement exfiltrées, vérifier les journaux d'accès, rechercher d'autres points d'injection), les correctifs long terme (audit de sécurité du code, formation des développeurs, mise en place de WAF) et les faux positifs (recherches légitimes contenant des apostrophes, noms avec des traits d'union).

5.6 Base de données

5.6.1 Schéma relationnel

La base de données minisoc_shop contient onze tables. Les tables métier de l'application sont : users (comptes clients et administrateurs), categories (catégories de produits), products (catalogue), reviews (avis clients), orders (commandes) et order_items (lignes de commande). Les tables de

sécurité sont : http_logs (logs collectés), alerts (alertes générées) et threat_score (scores de menace par IP). Les clés étrangères assurent l'intégrité référentielle entre products et categories, reviews et products, orders et users, order_items et orders. Les index optimisent les requêtes fréquentes : recherche par IP sur http_logs, recherche par type sur alerts, recherche par score sur threat_score.

5.6.2 Table alerts

La table alerts est la structure centrale du système de détection. Elle contient les champs : id (clé primaire), timestamp (datetime), log_id (référence vers http_logs), attack_type (SQLi, XSS, Brute Force), severity (CRITICAL, HIGH, MEDIUM, LOW), rule_id (identifiant de la règle déclenchée), description (texte explicatif), payload (le contenu malveillant détecté), source_ip (adresse IP de l'attaquant), url (page cible), mitre_id, mitre_tactic, mitre_technique, recommended_response (texte de remédiation), status (Nouveau, En cours, Résolu, Faux positif) et notes (champs libre pour l'analyste). Une contrainte d'unicité sur log_id, attack_type et source_ip évite les doublons d'alertes.

Chapitre 7 : Tests et Validation

6.1 Protocole de test

La phase de tests et de validation du Mini-SOC Web a pour objectif de vérifier que l'ensemble de la chaîne de détection et de réponse fonctionne correctement, de l'attaque initiale jusqu'à l'affichage de l'alerte dans le dashboard. Les tests sont organisés selon un protocole structuré qui couvre les trois types d'attaques, les différentes surfaces d'attaque de l'application et les scénarios de détection comportementale. Chaque test est documenté avec le payload utilisé, la page cible, le résultat attendu et le résultat observé.

L'environnement de test est un poste de travail local exécutant XAMPP avec Apache 2.4, PHP 8.2 et MySQL 8.0. Le Mini-SIEM est lancé en ligne de commande (python mini_siem.py) et le dashboard est accessible via le navigateur à l'adresse localhost/Mini_soc_complet/soc/. Le réseau est isolé (mode localhost uniquement) pour garantir la sécurité. Les comptes de test sont : admin/admin123, alice/123456, bob/password. Les mots de passe faibles sont intentionnels pour faciliter la démonstration de la force brute.

6.2 Tests d'injection SQL

6.2.1 Test sur la barre de recherche

Le premier test d'injection SQL vise la barre de recherche de la page search.php. Le payload saisi est : ' OR '1'='1. Ce payload ferme la chaîne de la clause LIKE, ajoute une condition OR 1=1 toujours vraie et ignore le reste de la requête. Résultat attendu : la page affiche l'intégralité du catalogue de produits, indépendamment du terme de recherche. Résultat observé : conforme à l'attendu. La page retourne tous les produits (AtlasPhone, AtlasBook, Caméra 360, SmartColor, PowerBank, Hub, etc.) avec le message de recherche affiché sans échappement, confirmant simultanément la vulnérabilité XSS réfléchi.

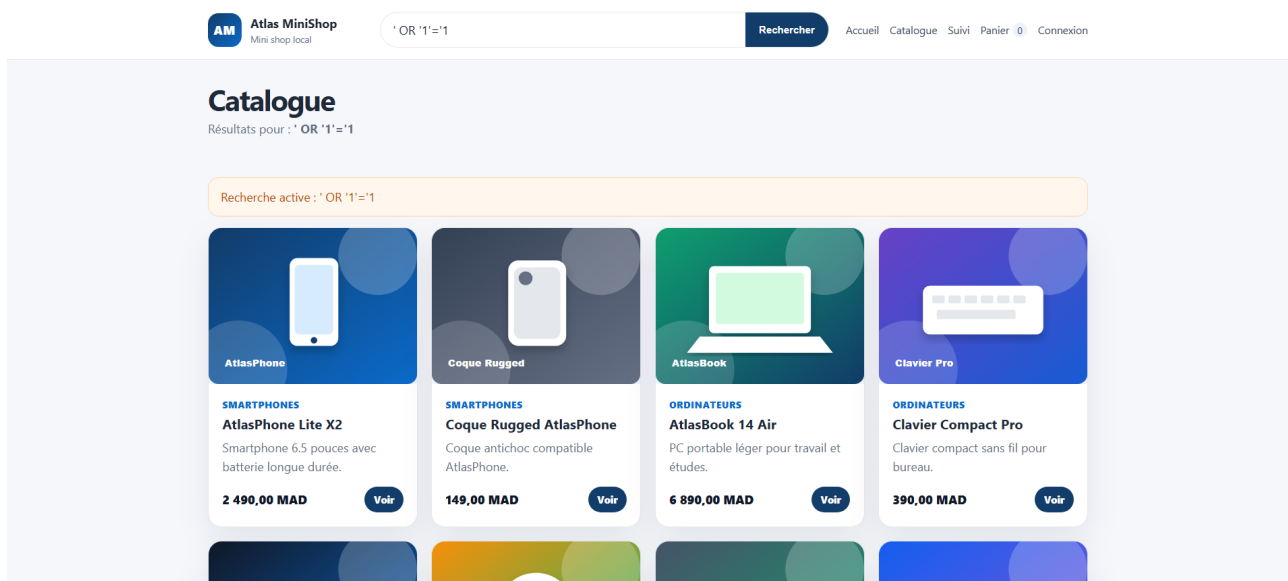


Figure 8.1 : Injection SQL dans la barre de recherche — payload 'OR '1'='1' retournant tout le catalogue

6.2.2 Test sur le formulaire de login

Le deuxième test cible le formulaire d'authentification de la page login.php. Le payload injecté dans le champ username est : admin' OR 1=1#. Ce payload ferme la chaîne du username, ajoute une condition OR 1=1 toujours vraie et commente le reste de la requête (vérification du mot de passe). Résultat attendu : connexion réussie en tant qu'administrateur sans connaître le mot de passe. Résultat observé : conforme. L'attaquant est redirigé vers la page admin.php avec les privilèges administrateur complets (gestion des commandes, des avis et des comptes).

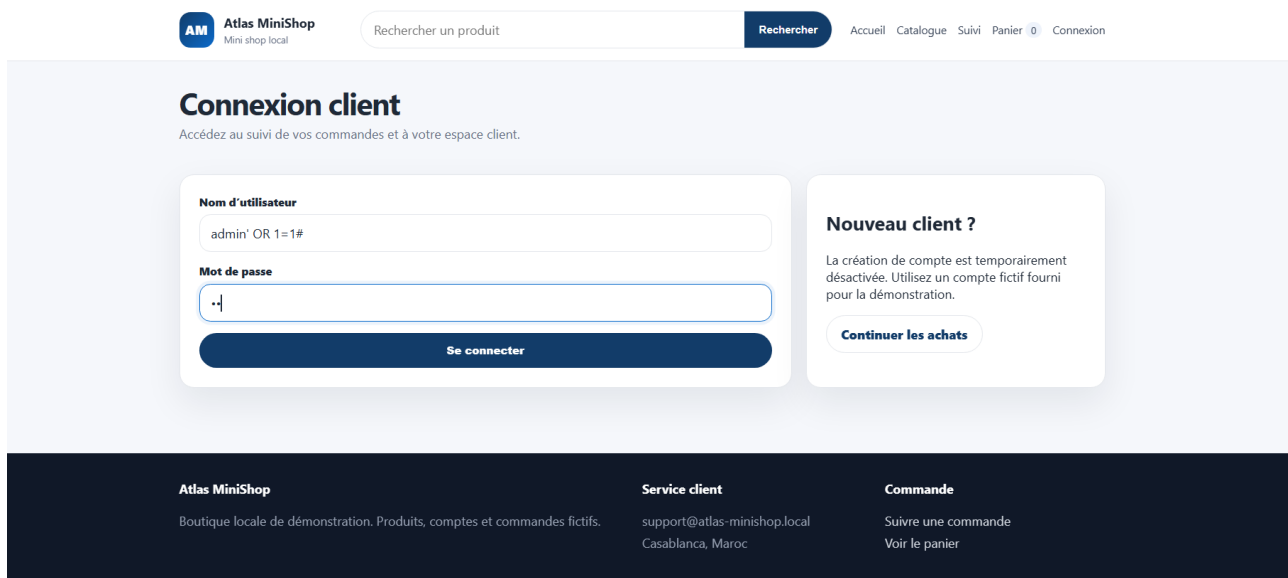


Figure 8.2 : Payload SQLi sur le login — admin' OR 1=1#

AM

Atlas MiniShop

Mini shop local

Rechercher un produit

Rechercher

[Accueil](#)
[Catalogue](#)
[Suivi](#)
[Panier 0](#)
[Compte](#)
[Admin](#)
[Sortir](#)

Mon compte

Bonjour Administrateur Atlas.

Profil

Utilisateur : admin

Email : admin@atlas-minishop.local

Rôle : admin

Mes commandes

Aucune commande enregistrée.

Atlas MiniShop
 Boutique locale de démonstration. Produits, comptes et commandes fictifs.

Service client
 support@atlas-minishop.local
 Casablanca, Maroc

Commande
[Suivre une commande](#)
[Voir le panier](#)

Figure 8.3 : Connexion réussie en tant qu'administrateur après injection SQL

6.2.3 Test de fuite d'informations par erreur SQL

Le troisième test exploite la mauvaise gestion des erreurs de l'application. Lorsqu'un payload malformé est soumis (admin' OR 1=1 suivi d'un caractère non attendu), la page affiche une erreur SQL brute contenant la requête complète exécutée. Cette fuite d'information révèle la structure de la requête, les noms de tables et de colonnes, facilitant l'élaboration d'attaques plus sophistiquées. Résultat observé : la page affiche `SELECT * FROM users WHERE username = 'admin' OR 1=1' AND password = 'sdfgs'`, confirmant la structure exacte de la requête vulnérable.

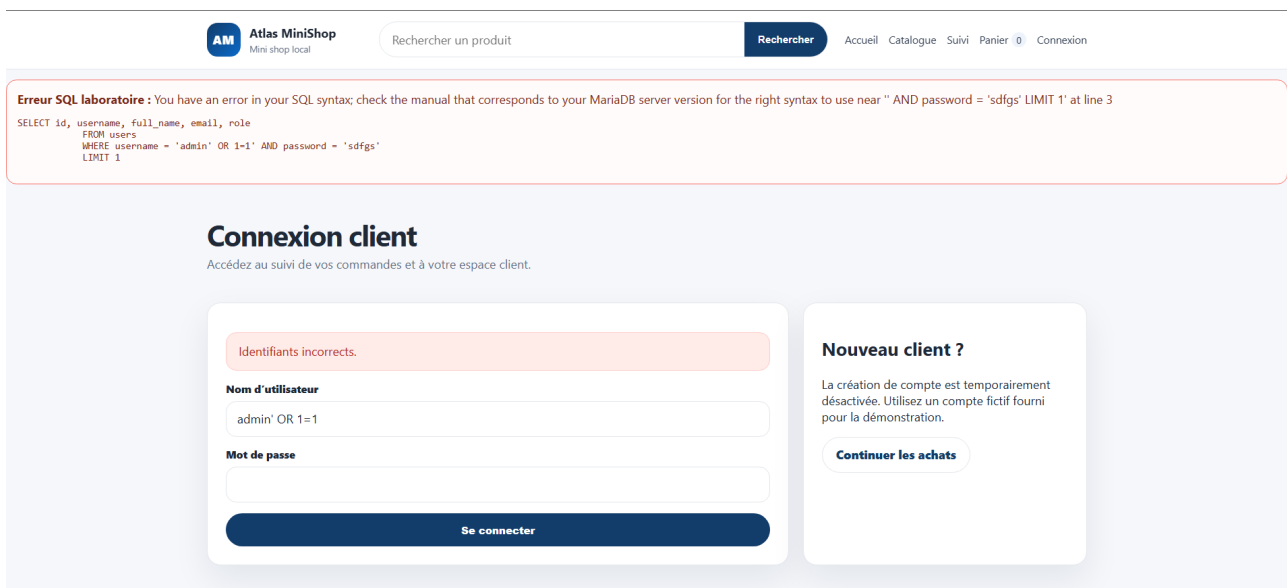


Figure 8.4 : Fuite d'information par erreur SQL — requête complète affichée en clair

6.2.4 Validation de la détection

Après chaque test SQLi, le Mini-SIEM a généré une alerte CRITICAL avec les informations suivantes : attack_type = SQLi, severity = CRITICAL, rule_id = SQLi-03, payload = admin' OR 1=1#, source_ip = 127.0.0.1, url = /Mini_soc_complet/login.php, mitre_id = T1190, mitre_tactic = Initial Access, mitre_technique = Exploit Public-Facing Application. Le dashboard a affiché l'alerte dans le tableau des alertes récentes avec le statut 'Nouveau'. La validation est donc totale : chaque attaque SQLi est détectée, classifiée, enrichie et affichée.

6.3 Tests de cross-site scripting

6.3.1 Test XSS réfléchi sur la recherche

Le test XSS réfléchi utilise le payload `<script>alert('XSS')</script>` dans la barre de recherche. Résultat attendu : une boîte de dialogue alert('XSS') s'affiche lors du chargement de la page de résultats. Résultat observé : conforme. Le navigateur exécute le script injecté car la variable `$_GET['q']` est affichée sans `htmlspecialchars()`. Le script a accès au cookie de session car il est configuré sans `HttpOnly`. Cette démonstration prouve la vulnérabilité XSS réfléchi et le risque de vol de session.



Figure 8.5 : XSS réfléchi dans la barre de recherche — payload alert('XSS')

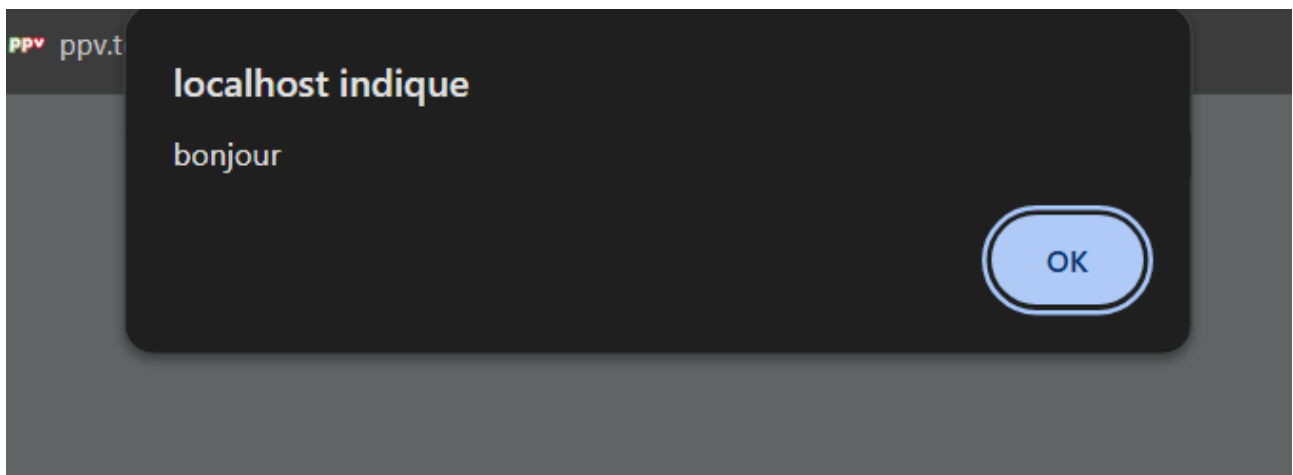


Figure 8.6 : Résultat XSS réfléchi dans le navigateur — Exécution du payload JavaScript

6.3.2 Test XSS stocké sur les avis

Le test XSS stocké utilise le payload `<script>alert('bonjour')</script>` dans le formulaire d'avis de la page `product.php`. Le commentaire est soumis par l'utilisateur alicia et enregistré dans la base de données. Résultat attendu : le script s'exécute pour chaque visiteur de la page produit, y compris l'administrateur. Résultat observé : conforme. Le navigateur affiche la boîte de dialogue `alert('bonjour')` car le champ `comment` est affiché sans échappement. La persistance du script dans la base de données en fait une menace plus grave que le XSS réfléchi.

Atlas MiniShop
Mini shop local

Rechercher un produit **Rechercher** Accueil Catalogue Suivi Panier 0 Connexion

0 avis sur ce produit.
Aucun avis pour le moment.

Nom
testeur

Note
5 étoiles

Commentaire
<script>alert('XSS')</script>

Publier

Atlas MiniShop
Boutique locale de démonstration. Produits, comptes et commandes fictifs.

Service client
support@atlas-minishop.local
Casablanca, Maroc

Commande
Suivre une commande
Voir le panier

Figure 8.7 : XSS stocké — formulaire d'avis avec payload alert('XSS')

phpMyAdmin

Base de données : minisoc_shop > Table : reviews

Parcourir Structure SQL Rechercher Insérer Exporter Importer Privileges Opérations Suivi Déclencheurs

Affichage des lignes 0 - 6 (total de 7, traitement en 0,0003 seconde(s).)

SELECT * FROM `reviews`

Profilage [Éditer en ligne] [Éditer] [Expliquer SQL] [Créer le code source PHP] [Actualiser]

Tout afficher Nombre de lignes : 25 Filtrer les lignes: Chercher dans cette table Trier par clé : Aucun(e)

Options supplémentaires

	id	product_id	author_name	rating	comment	created_at
☐ Éditer ☐ Copier ☐ Supprimer	1	1	Meryem	5	Très bon rapport qualité/prix pour une boutique de...	2026-05-13 14:43:42
☐ Éditer ☐ Copier ☐ Supprimer	2	1	Hamza	4	Design propre et livraison fictive rapide.	2026-05-15 14:43:42
☐ Éditer ☐ Copier ☐ Supprimer	3	3	Omar	5	Parfait pour les cours et les présentations.	2026-05-16 14:43:42
☐ Éditer ☐ Copier ☐ Supprimer	4	5	Salma	4	Interface claire, produit conforme à la descriptio...	2026-05-17 06:43:42
☐ Éditer ☐ Copier ☐ Supprimer	5	1	alice	5	très bon produit	2026-06-13 14:15:37
☐ Éditer ☐ Copier ☐ Supprimer	6	2	testeur	5	<script>alert("XSS")</script>	2026-06-13 14:28:09
☐ Éditer ☐ Copier ☐ Supprimer	7	2	testeur	5	<script>alert("bonjour")</script>	2026-06-13 14:30:31

Tout afficher Nombre de lignes : 25 Filtrer les lignes: Chercher dans cette table Trier par clé : Aucun(e)

Figure 8.8 : Table reviews sous phpMyAdmin — scripts XSS visibles en base de données

6.3.3 Validation de la détection XSS

Le Mini-SIEM a détecté les deux attaques XSS avec la règle XSS-01 (balise <script>). Les alertes générées sont de gravité HIGH avec les informations MITRE : T1059.007, Execution, Command and Scripting Interpreter : JavaScript. La réponse recommandée est : Encoder les sorties, filtrer les champs et vérifier les commentaires stockés. Le dashboard affiche correctement les alertes XSS avec le panneau de détail MITRE complet. La détection est validée.

6.4 Tests de force brute

6.4.1 Test de tentatives répétées

Le test de force brute consiste à soumettre cinq requêtes POST successives sur login.php avec des identifiants incorrects (aaa/bbb, ccc/dd, eee/fff, ggg/hhh, iii/jjj) depuis la même adresse IP 127.0.0.1 en moins de soixante secondes. Résultat attendu : le Mini-SIEM détecte le dépassement du seuil de cinq tentatives et génère une alerte HIGH. Résultat observé : conforme. La cinquième tentative déclenche l'alerte BF-01. Le dashboard affiche l'alerte Brute Force avec l'IP source, le nombre de tentatives et la fenêtre temporelle.

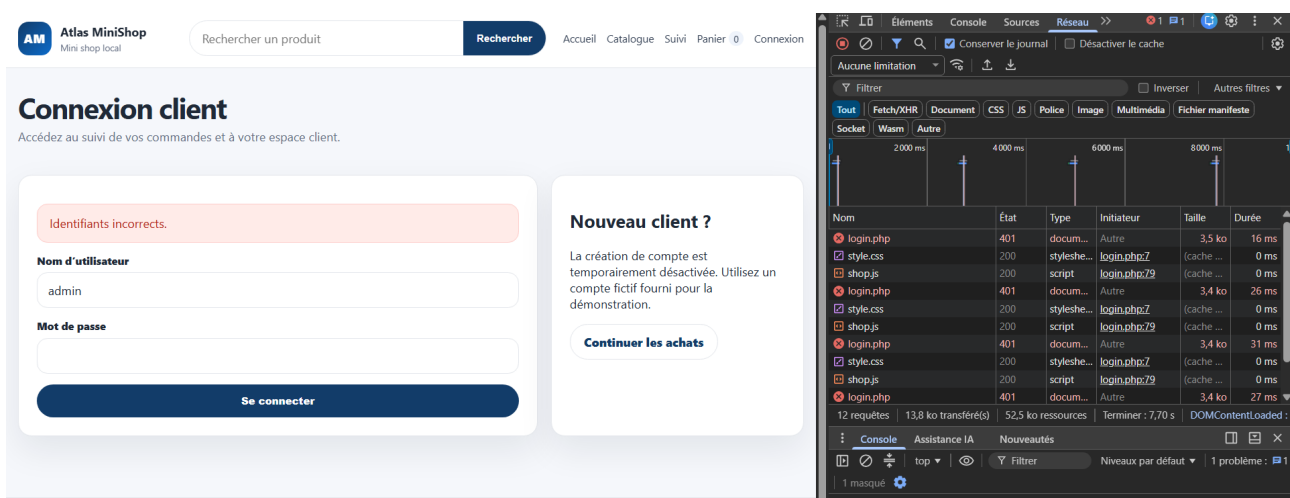


Figure 7.9 : Tentatives de force brute — outils de développement montrant les réponses 401 répétées

7.4.2 Validation de la détection Brute Force

L'alerte générée contient : attack_type = Brute Force, severity = HIGH, rule_id = BF-01, source_ip = 127.0.0.1, url = /Mini_soc_complet/login.php, mitre_id = T1110, mitre_tactic = Credential Access, mitre_technique = Brute Force. La réponse recommandée est : Analyser l'IP, limiter les tentatives et vérifier les comptes ciblés. Le score de menace de l'IP 127.0.0.1 est incrémenté de 8 points. La détection est validée.

7.5 Validation du cycle complet

La validation du cycle complet (end-to-end) vérifie que l'ensemble de la chaîne fonctionne de manière cohérente : attaque → log → détection → alerte → affichage → remédiation. Le scénario de validation est le suivant : l'attaquant soumet une injection SQL sur login.php, le collecteur logger.php capture la requête dans http_logs, le Mini-SIEM lit le log après 10 secondes, détecte

l'attaque avec la règle SQLi-03, charge le mapping MITRE, génère l'alerte enrichie dans la table alerts, le dashboard api.php rafraîchit les données et affiche la nouvelle alerte, l'administrateur consulte le détail et applique la réponse recommandée. Tous les scénarios de validation ont été exécutés avec succès. Le Mini-SOC Web est fonctionnel, reproductible et prêt pour la démonstration de soutenance.

Chapitre 8 : Dashboard SOC et Visualisation

8.1 Vue d'ensemble du dashboard

Le tableau de bord SOC constitue l'interface opérationnelle du Mini-SOC Web, c'est-à-dire le point central où l'analyste de sécurité interagit avec le système de détection. Il a été conçu selon les principes de l'ergonomie et de la lisibilité : les informations critiques doivent être visibles en un coup d'œil, les alertes doivent être contextualisées et actionnables, et la navigation doit être intuitive. Le design responsive assure une utilisation convenable sur différents formats d'écran.

La page d'accueil du dashboard est structurée en plusieurs zones fonctionnelles. La zone supérieure affiche les cartes de statistiques globales : le nombre total d'alertes, le nombre d'alertes par type (SQLi, XSS, Brute Force), le score de menace global et le nombre d'IP distinctes identifiées comme malveillantes. Ces cartes utilisent un code couleur pour indiquer la gravité : vert pour les faibles niveaux, orange pour les niveaux moyens et rouge pour les niveaux critiques.

La zone centrale présente les graphiques de visualisation. Le graphique de tendance temporelle montre l'évolution du nombre d'alertes sur les sept derniers jours, permettant d'identifier les pics d'activité et les patterns récurrents. Le diagramme de répartition par gravité (donut) présente la proportion des alertes CRITICAL, HIGH, MEDIUM et LOW. Le tableau du top 5 des IP attaquantes liste les adresses les plus actives avec leur score de menace associé.

La zone inférieure affiche le tableau des alertes récentes, avec les colonnes : type d'attaque, gravité, date et heure, IP source, URL cible et statut. Un clic sur une ligne ouvre le panneau de détail qui couvre la moitié droite de l'écran. Ce panneau présente l'ensemble des informations de l'alerte : payload complet, règle déclenchée, informations MITRE (ID, tactique, technique, réponse recommandée) et un formulaire de changement de statut.

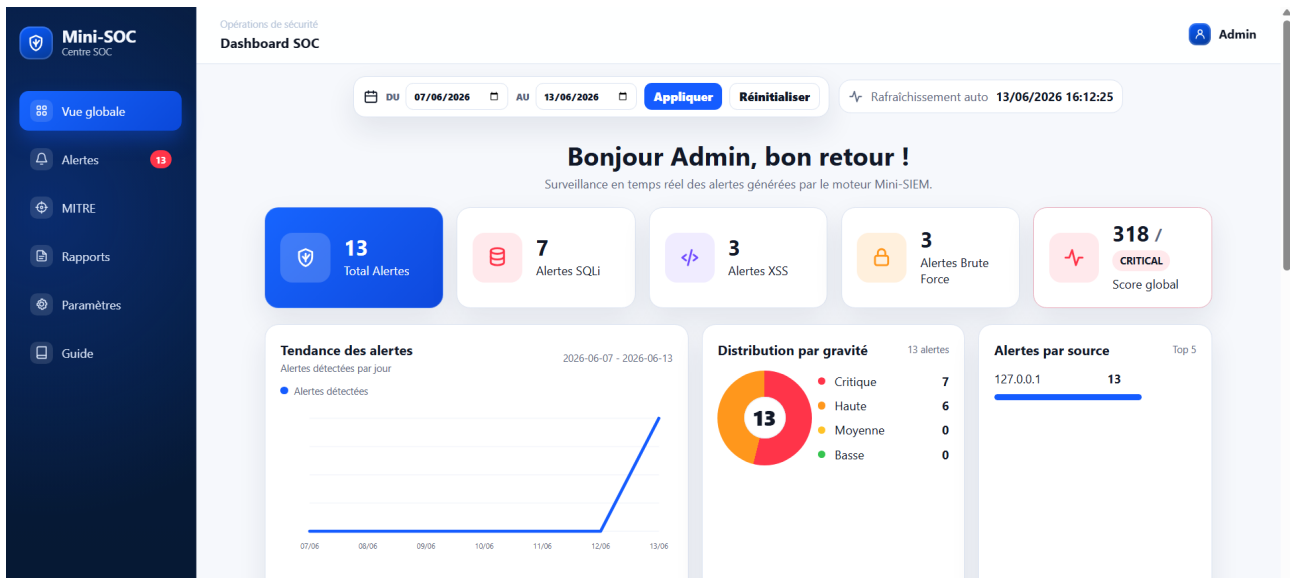


Figure 8.1 : Vue globale du dashboard SOC avec statistiques, graphiques et alertes récentes

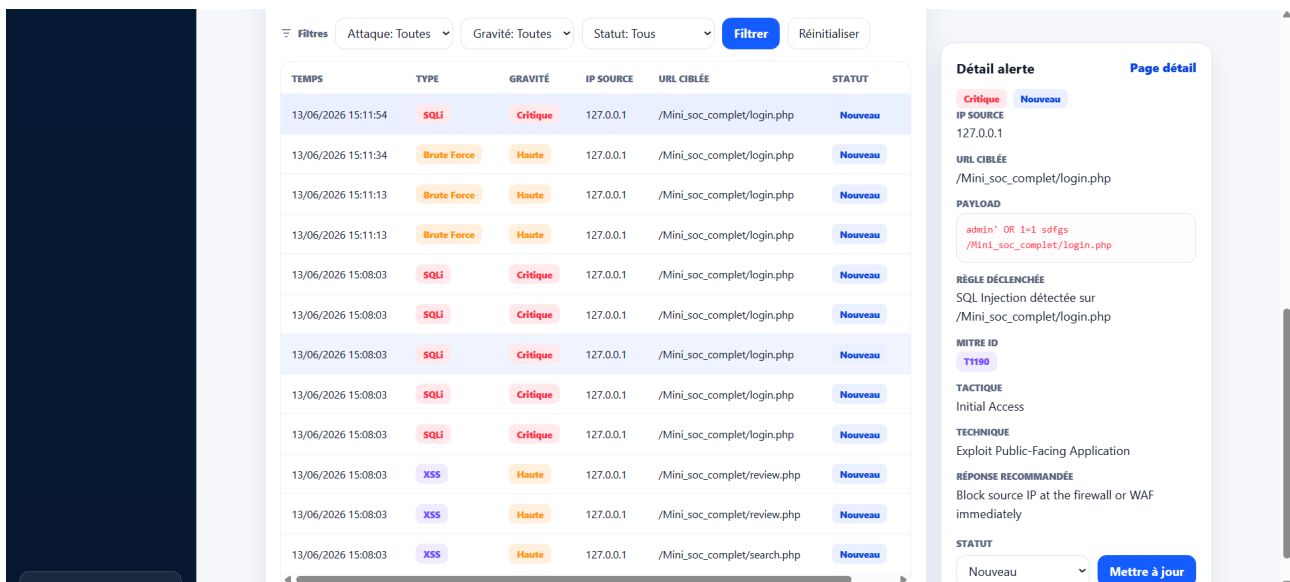


Figure 8.2 : Panneau de détail d'alerte avec informations MITRE complètes

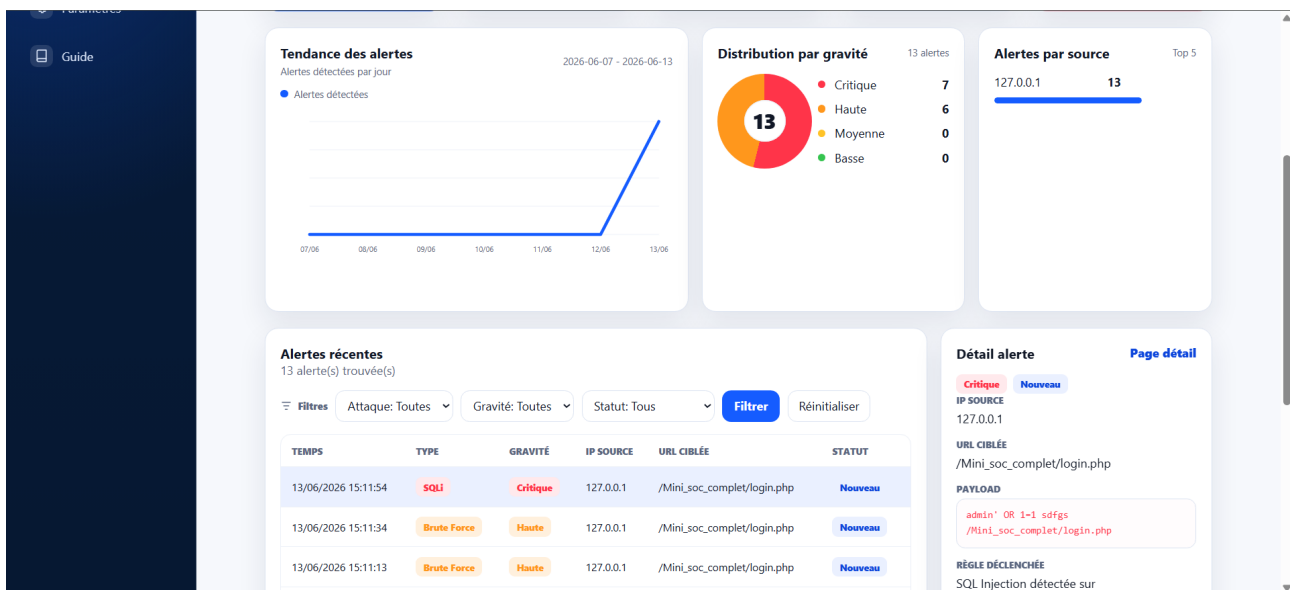


Figure 8.3 : Vue détaillée dashboard avec filtres et panneau de détail d'alerte

8.2 Statistiques et graphiques

8.2.1 Cartes de statistiques

Les cartes de statistiques sont des éléments visuels compacts qui affichent des indicateurs clés de performance (KPI) de la sécurité. Chaque carte est composée d'un titre, d'une valeur numérique, d'un indicateur de tendance (flèche haut/bas) et d'une couleur de fond adaptée à la gravité. La carte 'Total des alertes' affiche le nombre cumulé d'alertes depuis le début de la période de surveillance. La carte 'SQL Injection' affiche le nombre d'alertes SQLi avec un fond rouge si le nombre est supérieur à 5. La carte 'XSS' affiche le nombre d'alertes XSS. La carte 'Brute Force' affiche le nombre d'alertes de force brute. La carte 'Score de menace' affiche le score global avec une jauge colorée (vert 0-50, orange 51-100, rouge >100).

8.2.2 Graphique de tendance temporelle

Le graphique de tendance est un graphique en aires empilées qui montre l'évolution du nombre d'alertes par jour sur les sept derniers jours. L'axe horizontal représente les dates et l'axe vertical représente le nombre d'alertes. Les trois types d'attaque sont représentés par des couleurs distinctes : rouge pour SQLi, orange pour XSS et bleu pour Brute Force. Ce graphique permet d'identifier les journées à forte activité, les types d'attaque prédominants et les éventuelles corrélations temporelles. Le graphique est généré côté serveur sous forme de SVG inline pour une performance optimale et une résolution indépendante du zoom.



Figure 8.4 : Timeline des attaques — Graphique empilé montrant la répartition temporelle

8.2.3 Diagramme de répartition par gravité

Le diagramme de répartition par gravité est un graphique en anneau (donut) qui présente la proportion des alertes selon leur niveau de sévérité. Les quatre segments correspondent aux niveaux CRITICAL (rouge foncé), HIGH (rouge), MEDIUM (orange) et LOW (vert). Le pourcentage de chaque segment est affiché au centre du donut. Ce graphique permet d'évaluer rapidement la criticité globale du paysage de menaces : un donut dominé par le rouge indique une situation critique nécessitant une intervention immédiate.

8.3 Fiches de détail d'alerte

8.3.1 Structure du panneau de détail

Le panneau de détail est un composant modal qui s'affiche lors du clic sur une alerte dans le tableau. Il est divisé en quatre sections : (1) Informations générales — type d'attaque, gravité, date, IP source, URL cible, payload complet ; (2) Informations MITRE — ID de technique, tactique, technique, description ; (3) Réponse recommandée — actions immédiates à entreprendre ; (4) Gestion du statut — formulaire permettant de changer le statut de l'alerte (Nouveau, En cours, Résolu, Faux positif) et d'ajouter des notes. Le panneau utilise une typographie claire avec des titres en gras, des codes en monospace et des blocs de citation pour les réponses recommandées.

MITRE ATT&CK utilisé
Explication des mappings MITRE intégrés au Mini-SIEM pour ce projet Mini-SOC.

SQL Injection T1190
 search.php, login.php, product.php, track_order.php
TACTIQUE
Initial Access
TECHNIQUE
Exploit Public-Facing Application
GRAVITÉ PAR DÉFAUT
Critique
SURFACE PROJET
search.php, login.php, product.php, track_order.php
UTILISATION DANS LE MINI-SIEM
Le Mini-SIEM inspecte les paramètres GET/POST et les User-Agent suspects pour détecter les mots clés SQL, UNION SELECT, commentaires SQL, payloads booléens et time-based.
RÉPONSE RECOMMANDÉE
Block source IP at the firewall or WAF immediately

Cross-Site Scripting T1059.007
 review.php, search.php, track_order.php
TACTIQUE
Execution
TECHNIQUE
Command and Scripting Interpreter: JavaScript
GRAVITÉ PAR DÉFAUT
Haute
SURFACE PROJET
review.php, search.php, track_order.php
UTILISATION DANS LE MINI-SIEM
Le moteur recherche les balises script, handlers JavaScript, javascript; SVG/IMG onload/onclick et payloads encodés dans les champs utilisateur.

Brute Force T1110
 login.php
TACTIQUE
Credential Access
TECHNIQUE
Brute Force
GRAVITÉ PAR DÉFAUT
Haute
SURFACE PROJET
login.php
UTILISATION DANS LE MINI-SIEM
Le moteur compte les échecs de connexion par IP dans une fenêtre courte et enrichit l'alerte avec la technique MITRE Credential Access.
RÉPONSE RECOMMANDÉE

Figure 8.5 : Page MITRE ATT&CK; — Mapping complet des trois types d'attaque

8.3.2 Informations MITRE intégrées

L'intégration MITRE dans le panneau de détail est un atout majeur du projet. Elle transforme une alerte technique brute en un incident de sécurité documenté selon les standards internationaux. L'affichage MITRE comprend : le badge de l'identifiant de technique (cliquable pour ouvrir la page MITRE officielle), la description de la tactique, la description détaillée de la technique, la liste des indicateurs de compromission observés, les étapes d'investigation recommandées et les liens vers les ressources MITRE. Cette contextualisation permet à l'analyste de comprendre non seulement CE QUI s'est passé, mais aussi COMMENT et POURQUOI, et surtout COMMENT RÉAGIR.

8.4 Rapports PDF et documentation

8.4.1 Génération de rapports natifs

Le module de génération de rapports est une fonctionnalité avancée qui produit des documents PDF directement depuis le dashboard, sans dépendance externe. Le script report.php construit le PDF en assemblant manuellement les objets du format PDF 1.4 : en-tête, catalogue, pages, flux de contenu, images et références croisées. Le rapport généré contient : une page de couverture avec le titre Mini-SOC Report, la date et la période couverte ; un résumé exécutif avec les statistiques globales (total, répartition par type, score moyen) ; une analyse détaillée par type d'attaque avec les recommandations MITRE ; une timeline des incidents sous forme de tableau chronologique ; et un glossaire des termes techniques. Le rapport est téléchargeable et imprimable, offrant une traçabilité documentaire des opérations de sécurité.

Mini-SOC
Centre SOC

Opérations de sécurité
Rapport SOC PDF

Rapport SOC PDF
Générer un rapport de synthèse clair à partir des alertes Mini-SIEM filtrées.

Paramètres du rapport

Date début: 07/06/2026

Date fin: 13/06/2026

Type attaque: Toutes

Gravité: Toutes

Statut: Tous

Prévisualiser

2026-06-07 - 2026-06-13

Total: 13

Score global: 318 / CRITICAL

SQLi: 7 (53.8%)

XSS: 3 (23.1%)

Brute Force: 3 (23.1%)

Pourcentages des attaques

Attaque dominante: SQLi 53.8%

SQLi: 7 - 53.8%

XSS: 3 - 23.1%

Brute Force: 3 - 23.1%

SQLi

7 cas - Dernier signal 13/06/2026 15:08:03

T1190 Initial Access - Exploit Public-Facing Application

Sources principales: 127.0.0.1

Analyse incident: Tentatives SQLi detectées sur des pages publiques. Le risque principal est l'accès non autorisé aux données, le contournement de l'authentification ou la modification de la base.

REPONSE RECOMMANDÉE

Figure 8.6 : Interface de génération de rapport PDF — Paramètres et prévisualisation

8.4.2 Guide d'utilisation intégré

Le guide d'utilisation (guide.php) est une documentation opérationnelle complète intégrée au dashboard. Il présente le processus de gestion des incidents en quatre étapes clairement définies : (1) Préparer le laboratoire — instructions pour lancer l'application vulnérable, démarrer le Mini-SIEM et ouvrir le dashboard ; (2) Activer la détection — vérification du fonctionnement du collecteur de logs et du moteur de détection ; (3) Lire le dashboard — interprétation des alertes, compréhension des informations MITRE et évaluation de la gravité ; (4) Traiter et documenter — application des réponses recommandées, changement de statut des alertes et génération de rapports. Le guide est rédigé dans un langage clair et accessible, avec des captures d'écran, des astuces et des avertissements. Il constitue une ressource pédagogique autonome pour les utilisateurs du Mini-SOC.

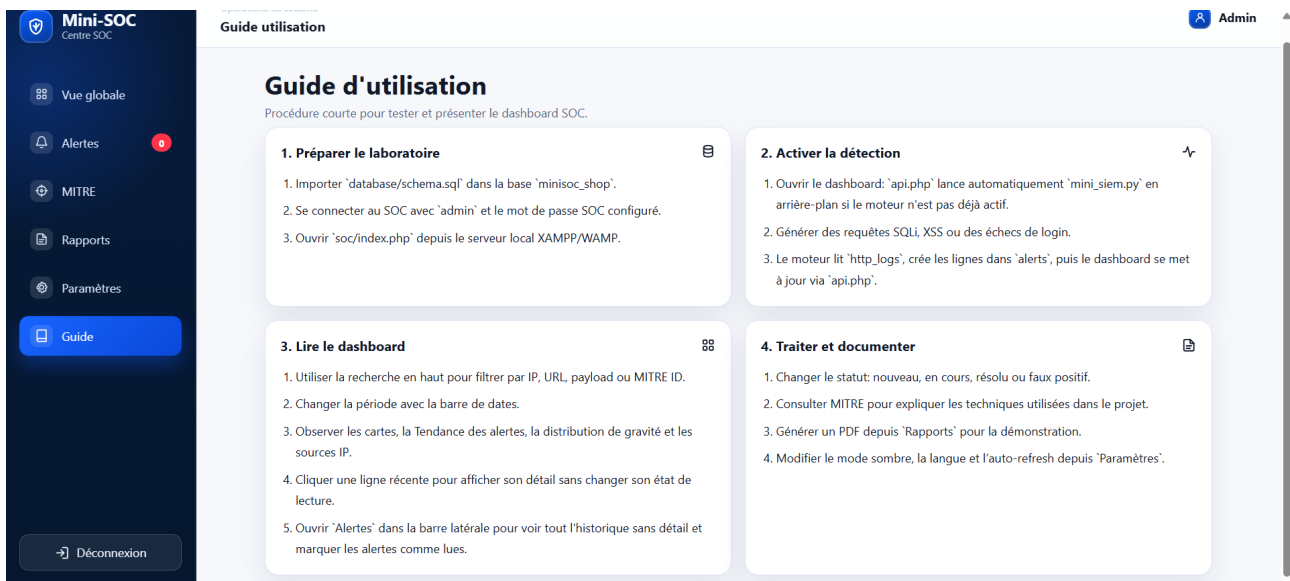


Figure 8.7 : Guide d'utilisation — Processus de gestion des incidents en 4 étapes

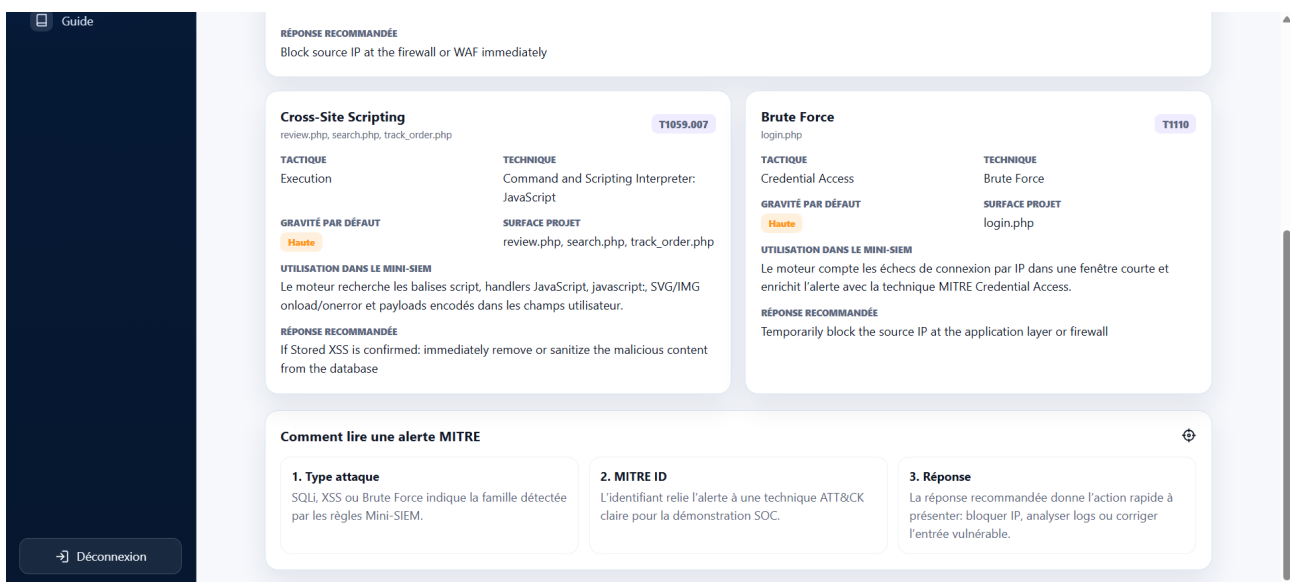


Figure 8.8 : Réponses recommandées MITRE — Actions correctives pour chaque attaque

8.5 Configuration et personnalisation

8.5.1 Paramètres utilisateur

La page settings.php permet à l'administrateur de personnaliser l'expérience du dashboard. Les paramètres configurables sont : le thème visuel (clair ou sombre), la langue de l'interface (français ou anglais), la taille du tableau d'alertes (10, 25, 50 ou 100 lignes par page), le tri par défaut (date décroissante, gravité décroissante), les filtres de sévérité affichés par défaut et l'affichage des informations MITRE dans le tableau. Les préférences sont stockées dans le localStorage du

navigateur et appliquées immédiatement sans rechargement de page. La page permet également le changement du mot de passe administrateur avec vérification de la complexité.

8.5.2 Mode confidentialité

Le mode confidentialité est une fonctionnalité de protection des données sensibles. Lorsqu'il est activé, les adresses IP, les payloads et les URLs sont masqués dans le tableau des alertes (affichage sous forme de hash ou d'astérisques). Seul l'administrateur authentifié peut désactiver ce mode pour consulter les données complètes. Cette fonctionnalité est particulièrement utile lors de démonstrations publiques ou de captures d'écran, où la divulgation d'informations sensibles doit être évitée.

Conclusion générale

Le projet Mini-SOC Web représente une réalisation académique complète et cohérente qui démontre l'ensemble du cycle de vie d'une opération de sécurité informatique, de l'attaque initiale jusqu'à la remédiation documentée. Au travers de cinq modules interconnectés — une application web vulnérable, un collecteur de logs, un moteur de détection, un tableau de bord SOC et un mapping MITRE ATT&CK; — nous avons construit un laboratoire pédagogique fonctionnel, répliquable et extensible.

Les objectifs fixés au début du projet ont été atteints avec succès. L'application Atlas MiniShop offre un terrain d'expérimentation réaliste avec trois vulnérabilités représentatives (injection SQL, cross-site scripting et force brute). Le collecteur de logs assure une acquisition exhaustive et transparente des événements HTTP. Le Mini-SIEM détecte automatiquement les attaques avec des règles regex et des seuils comportementaux, génère des alertes classifiées et calcule un score de menace. Le dashboard SOC fournit une interface opérationnelle complète avec visualisation temps réel, détails techniques, enrichissement MITRE et génération de rapports. Le mapping MITRE ATT&CK; apporte une dimension professionnelle et académique en alignant le projet avec les standards internationaux de la cybersécurité.

Sur le plan technique, le projet met en œuvre une stack moderne et accessible : PHP 8.x, MySQL 8.x, Python 3.10+, HTML5, CSS3 et JavaScript. L'architecture modulaire facilite la maintenance, l'évolution et la réutilisation des composants. La sécurité du dashboard est renforcée par des mécanismes de protection contre les attaques classiques (CSRF, hachage bcrypt, limitation des tentatives, en-têtes CSP). La génération de rapports PDF natifs et le guide d'utilisation intégré en font un outil autonome et pédagogiquement complet.

Sur le plan organisationnel, la répartition des tâches a permis à chaque membre de l'équipe de contribuer de manière équilibrée et complémentaire. Les dépendances entre les modules ont été respectées, et la collaboration entre les membres a assuré une intégration harmonieuse. Le chef de projet a coordonné l'ensemble, assuré la cohérence des livrables et préparé la démonstration finale. Les phases de test et de validation ont confirmé la fiabilité du système dans des scénarios réels d'attaque.

Le projet ouvre plusieurs perspectives d'amélioration et d'extension. L'intégration de capacités de machine learning pour la détection d'anomalies comportementales représente une évolution naturelle. Le passage à une architecture distribuée avec Elasticsearch pour le stockage des logs et Kafka pour la messagerie permettrait de traiter des volumes plus importants. L'ajout de notifications par email ou SMS, d'une intégration avec des plateformes de Threat Intelligence et d'un module

SOAR pour l'orchestration automatique des réponses élèverait le projet au niveau d'un SOC professionnel. Enfin, le déploiement sur un environnement cloud avec conteneurisation Docker faciliterait la diffusion et la réplication du projet.

En conclusion, le Mini-SOC Web constitue une démonstration tangible de l'application des concepts théoriques de la cybersécurité dans un contexte pratique et contrôlé. Il illustre la complémentarité entre la compréhension offensive (comment attaquer) et la compréhension défensive (comment détecter et répondre), qui est au cœur de la formation en cybersécurité. Nous espérons que ce projet servira de référence et d'inspiration pour les promotions futures de la filière cybersécurité de l'EMSI.

Glossaire

SIEM (Security Information and Event Management) : Plateforme qui centralise la collecte, la corrélation et l'analyse des logs de sécurité pour détecter les incidents en temps réel.

SOC (Security Operations Center) : Centre d'opérations de sécurité chargé de surveiller, détecter, analyser et répondre aux cybermenaces sur l'infrastructure d'une organisation.

SQL Injection (SQLi) : Attaque consistant à injecter du code SQL malveillant dans une requête pour manipuler une base de données et accéder à des données non autorisées.

XSS (Cross-Site Scripting) : Vulnérabilité permettant d'injecter du code JavaScript malveillant dans une page web vue par d'autres utilisateurs, pour voler des sessions ou rediriger vers des sites frauduleux.

Force Brute (Brute Force) : Méthode d'attaque qui teste systématiquement un grand nombre de combinaisons de mots de passe pour compromettre un compte utilisateur.

MITRE ATT&CK : Framework de connaissances qui recense les tactiques et techniques utilisées par les attaquants, servant de référence pour la détection et la réponse aux incidents.

Payload : Donnée malveillante injectée par un attaquant dans une application web (ex : ' OR 1=1# pour SQLi, <script>alert('XSS')</script> pour XSS).

Regex (Expression régulière) : Pattern de texte utilisé pour détecter des motifs suspects dans les logs HTTP.

Log HTTP : Enregistrement d'une requête web contenant IP, URL, méthode, paramètres, User-Agent et code de réponse.

Alerte : Notification générée par le Mini-SIEM lorsqu'une attaque est détectée, enrichie avec les informations MITRE.

Threat Score : Score numérique calculé par le Mini-SIEM pour quantifier le niveau de menace d'une IP attaquante.

Faux positif : Alerte déclenchée par erreur sur un comportement légitime (à minimiser par les règles de détection).

Faux négatif : Attaque réelle non détectée par le système (à minimiser par les règles de détection).

Token CSRF : Jeton de sécurité unique par session utilisé pour prévenir les attaques de type Cross-Site Request Forgery.

Content Security Policy (CSP) : En-tête HTTP qui contrôle les sources de contenu exécutables sur une page web.

Hachage bcrypt : Algorithme de hachage de mots de passe avec salage automatique, utilisé pour sécuriser le dashboard SOC.

HttpOnly : Attribut de cookie qui empêche l'accès au cookie via JavaScript, protégeant contre le vol de session XSS.

Session fingerprinting : Technique de sécurité qui lie une session utilisateur à son navigateur et son IP pour détecter les détournements.

Playbook : Guide opérationnel documentant les étapes de réponse à un incident de sécurité spécifique.

Threat Intelligence : Connaissance contextualisée sur les menaces et les attaquants utilisée pour enrichir la détection.

Corrélation : Analyse croisée de plusieurs événements de sécurité pour identifier des patterns d'attaque complexes.

Incident Response : Processus structuré de réponse à un incident de sécurité, de la détection à la remédiation.

Wazuh : Plateforme open-source de sécurité qui combine HIDS, EDR et conformité, souvent intégrée à la stack ELK pour constituer un SOC complet.

ELK Stack (Elasticsearch, Logstash, Kibana) : Suite open-source de centralisation et de visualisation de logs, très utilisée dans les infrastructures de sécurité.

DVWA (Damn Vulnerable Web Application) : Application web volontairement vulnérable utilisée comme terrain d'apprentissage pour la cybersécurité offensive et défensive.

IBM QRadar : Plateforme SIEM entreprise d'IBM qui intègre la gestion des vulnérabilités, l'analyse des flux réseau et la corrélation automatique des événements.

Microsoft Sentinel : Solution cloud-native SIEM et SOAR de Microsoft, intégrée à Azure, combinant intelligence artificielle et threat hunting.

Bibliographie

Ouvrages et articles

[1] OWASP Foundation. OWASP Top 10 : 2021. OWASP, 2021. Disponible sur : <https://owasp.org/www-project-top-ten/>

[2] MITRE Corporation. MITRE ATT&CK; Framework, version 14. MITRE, 2024. Disponible sur : <https://attack.mitre.org/>

[3] Splunk Inc. Splunk Enterprise Security Documentation. Splunk, 2024. Disponible sur : <https://docs.splunk.com/Documentation/ES>

[4] IBM Corporation. IBM QRadar SIEM User Guide. IBM, 2024. Disponible sur : <https://www.ibm.com/docs/en/qradar-common>

[5] Microsoft Corporation. Microsoft Sentinel Documentation. Microsoft Learn, 2024. Disponible sur : <https://learn.microsoft.com/azure/sentinel/>

[6] Elastic NV. Elastic Stack Documentation : Elasticsearch, Logstash, Kibana. Elastic, 2024. Disponible sur : <https://www.elastic.co/guide/>

[7] Wazuh Team. Wazuh Documentation. Wazuh, 2024. Disponible sur : <https://documentation.wazuh.com/>

[8] TrendyWall. DVWA : Damn Vulnerable Web Application. GitHub, 2024. Disponible sur : <https://github.com/digininja/DVWA>

[9] OWASP Foundation. WebGoat Project. OWASP, 2024. Disponible sur : <https://owasp.org/www-project-webgoat/>

[10] ModSecurity Team. ModSecurity Handbook. Feisty Duck, 2022. ISBN : 978-1907117030

[11] S. Northcutt, L. Zeltser, et al. Securing Network Infrastructure. SANS Institute, 2020.

[12] R. Bejtlich. The Practice of Network Security Monitoring : Understanding Incident Detection and Response. No Starch Press, 2013. ISBN : 978-1593275099

[13] J. R. Vacca. Computer and Information Security Handbook. Morgan Kaufmann, 2017. ISBN : 978-0128038437

[14] A. Shostack. Threat Modeling : Designing for Security. Wiley, 2014. ISBN : 978-1118810057

[15] National Institute of Standards and Technology (NIST). Cybersecurity Framework, version 1.1. NIST, 2018. Disponible sur : <https://www.nist.gov/cyberframework>

[16] ISO/IEC 27001 : 2022. Information Security, Cybersecurity and Privacy Protection — Information Security Management Systems — Requirements. ISO, 2022.

[17] K. Scarfone, P. Mell. Guide to Intrusion Detection and Prevention Systems (IDPS). NIST Special Publication 800-94, 2012. Disponible sur : <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-94.pdf>

Webographie

- OWASP Top 10 — <https://owasp.org/www-project-top-ten/>
- MITRE ATT&CK; Framework — <https://attack.mitre.org/>
- Splunk Enterprise Security Documentation — <https://docs.splunk.com/Documentation/ES>
- IBM QRadar SIEM Documentation — <https://www.ibm.com/docs/en/qradar-common>
- Microsoft Sentinel Documentation — <https://learn.microsoft.com/azure/sentinel/>
- Elastic Stack Documentation — <https://www.elastic.co/guide/>
- Wazuh Documentation — <https://documentation.wazuh.com/>
- DVWA GitHub Repository — <https://github.com/digininja/DVWA>
- WebGoat OWASP Project — <https://owasp.org/www-project-webgoat/>
- NIST Cybersecurity Framework — <https://www.nist.gov/cyberframework>
- NIST SP 800-94 (IDPS Guide) — <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-94.pdf>
- ISO/IEC 27001 — <https://www.iso.org/isoiec-27001-information-security.html>